

Actividad 2 - Documento de formulación del proyecto

Santiago José Barbosa Rivas ID 100198965

Jerson Javier Ramírez Ricardo ID 100123048

Mario Alexander Avellaneda Buitrago ID 100180605

José Luis Arias ID 100143942

Facultad De Ingeniería, Programa De Ingeniería De Software

Corporación Universitaria Iberoamericana

Proyecto de software

Dra. Tatiana Cabrera

05 De Abril De 2025

Tabla de contenido

Introducción.....	5
Desarrollo del trabajo	6
Identificación de la problemática:	6
Contexto de la problemática:	6
Población afectada:	7
Justificación de la solución tecnológica:	7
Objetivos del proyecto:	8
Alcance del proyecto:	9
Alcance funcional del sistema:	9
Tabla 1: <i>modulo funcional del sistema “Vecino”</i>	9
Tabla 2: <i>aspectos que no estarán en el proyecto “Vecino”</i>	10
Beneficios esperados del sistema:	11
Tabla 3: <i>beneficios esperados del sistema</i>	11
Metodología de desarrollo:	12
Roles del equipo:	12
Tabla 4: <i>roles del equipo</i>	13
Organización del trabajo mediante Sprints:	13
Herramientas de gestión:.....	14
Figura 1: <i>tablero Jira, gestión de historias de usuario y sprints del proyecto Vecino</i>	14
Figura 2: <i>tablero de gestión del proyecto Vecino en Jira, Backlog con historias de usuario organizadas por Sprint</i>	15
.....	15
Figura 3: <i>Git Hub del repositorio del BackEnd</i>	15
Figura 4: <i>Git Hub del repositorio del FrontEnd</i>	16
Requisitos del sistema:	16
Tabla 5: <i>requisitos funcionales</i>	17

Tabla 6: requisitos no funcionales	17
Identificación de stakeholders y usuarios finales:	19
Tabla 7: <i>identificación de stakeholders y usuarios finales</i>	19
Tabla 8: <i>historia de usuario del sistema Vecino</i>	20
Flujos de Solución	23
Figura 5: <i>flujo general del sistema Vecino según rol del usuario</i>	23
Figura 6: <i>proceso de compra del consumidor</i>	23
Figura 7: <i>interacción del comerciante con el sistema</i>	24
Figura 8: <i>diagrama de casos de uso del sistema Vecino</i>	24
Figura 9: <i>diagrama entidad-relación del sistema Vecino</i>	25
Figura 10: <i>diagrama de secuencia, flujo de compra del consumidor en el sistema Vecino</i>	26
Solución tecnológica a implementar	27
Descripción del sistema:	27
Arquitectura general de la solución:	27
Figura 11: <i>arquitectura general de la solución Vecino</i>	28
Tecnologías que se utilizarán:	28
Tabla 9: <i>tecnologías que se utilizaran en el sistema Vecino</i>	28
Beneficios esperados del sistema:	29
Modelamiento del sistema	30
Figura 12: <i>diagrama de clases del sistema Vecino</i>	30
Figura 13: <i>prototipo de alta fidelidad, registro de usuario</i>	31
Figura 14: <i>prototipo de alta fidelidad, recuperación de contraseña</i>	31
Figura 15: <i>prototipo de alta fidelidad, inicio de sesión</i>	32
Figura 16: <i>prototipo de alta fidelidad, gestión de negocios</i>	32
Figura 17: <i>prototipo de alta fidelidad, catálogo de productos</i>	33
Figura 18: <i>prototipo de alta fidelidad, motor de búsqueda y filtros</i>	33

Figura 19: <i>prototipo de alta fidelidad, carrito de compras</i>	34
Figura 20: <i>prototipo de alta fidelidad, registro de pedido</i>	34
Figura 21: <i>prototipo de alta fidelidad, confirmación de pedido</i>	35
Figura 22: <i>prototipo de alta fidelidad, seguimiento de pedido</i>	35
Figura 23: <i>prototipo de alta fidelidad, pasarela de pagos</i>	36
Figura 24: <i>prototipo de alta fidelidad, sistema de calificaciones</i>	36
Figura 25: <i>prototipo de alta fidelidad, sistema de reseñas</i>	37
Figura 26: <i>prototipo de alta fidelidad, módulo de geolocalización</i>	37
Figura 27: <i>prototipo de alta fidelidad, panel del comerciante</i>	38
Figura 28: <i>prototipo de alta fidelidad, administración del sistema</i>	38
Conclusiones	39
Referencias	40

Introducción

Para empezar, el modo de acceder a productos y servicios en la vida diaria ha cambiado de manera significativa gracias a la transformación digital. No obstante, en las ciudades intermedias de Colombia, los comerciantes menores y empresarios locales siguen encontrando obstáculos importantes para incorporarse a la economía digital, sobre todo debido a los elevados precios de las plataformas disponibles y la ausencia de herramientas tecnológicas asequibles. Este documento explica cómo se formula, diseña y planifica un proyecto de software llamado Marketplace Hiperlocal. El objetivo es vincular a los consumidores, emprendedores y comerciantes en una misma área geográfica y hacer más fácil la compra y venta de productos y servicios locales a través de una plataforma digital que se puede usar por medio de la web.

Desarrollo del trabajo

Identificación de la problemática:

En el marco del comercio urbano de Colombia, los negocios pequeños, los emprendedores y los productores locales tienen cada vez más problemas para hacerse visibles y competir en el entorno digital. Aunque hoy en día hay varias plataformas de comercio electrónico, éstas funcionan con modelos que favorecen a las empresas con mayor capacidad económica, lo cual deja a los comercios más pequeños en una situación estructuralmente desventajosa. Además, una gran mayoría de estas empresas no cuentan con herramientas tecnológicas asequibles para promover y comercializar sus productos en línea de forma autónoma y sostenible.

Desde el punto de vista del cliente, la situación no es menos problemática. Los usuarios tienden a utilizar con frecuencia aplicaciones de gran difusión para efectuar sus compras, sin saber que en su propio entorno próximo tienen disponibles otras ofertas. Esta dinámica deteriora la economía local de manera gradual y destruye las conexiones naturales entre los residentes de una comunidad y sus comerciantes más cercanos.

Contexto de la problemática:

Ahora bien, la situación que se describe pertenece al sector de servicios y comercio, en particular a las economías locales de ciudades intermedias colombianas. En estos contextos, prevalecen las pequeñas empresas informales o en fase de expansión, que se distinguen por su escasa digitalización y la adopción restringida de herramientas tecnológicas. Esta condición se debe sobre todo a los elevados costes de implementación y a la falta de conocimiento acerca de las soluciones que existen en el mercado.

Esta brecha digital, además de afectar la economía, tiene efectos sociales: disminuye el sostén a la economía barrial, se hace más dependiente de plataformas externas que brindan visibilidad, pero no ayudan a robustecer el tejido productivo local y se debilita la relación entre los comerciantes y los habitantes del barrio.

Población afectada:

Por consiguiente, los comerciantes y emprendedores locales son quienes, en primer término, se ven más perjudicados por esta situación: estos incluyen tiendas de vecindario, restaurantes de pequeño tamaño como las cocinas ocultas, vendedores independientes y productores artesanales que no tienen medios digitales propios para hacer publicidad y comercializar sus productos. En segundo lugar, los clientes se ven afectados; estos son las personas que viven en barrios y comunidades que, debido a la falta de una plataforma que reúna la oferta local, terminan utilizando aplicaciones masivas ajenas a su entorno inmediato.

Además, hay actores indirectos que también se ven afectados, como los organizadores de eventos o ferias comunitarias y los repartidores locales, quienes tendrían la posibilidad de sacar provecho de una mejor coordinación digital entre los participantes del comercio local.

Justificación de la solución tecnológica:

Asimismo, el desarrollo de una plataforma digital tipo mercado en línea hiperfocal es una solución factible, que puede ser ampliada y tiene un gran impacto tanto a nivel económico como social, frente a la problemática detectada. Un instrumento como este permitiría la geolocalización de la oferta local para que los usuarios puedan hallar productos y servicios cercanos de forma eficaz. Además, les ofrece a los minoristas pequeños un medio asequible para vender en línea sin tener que pagar por las plataformas tradicionales, que son muy costosas.

En términos de empoderamiento comunitario, la plataforma fomentaría el consumo dentro del misma área al crear confianza entre compradores y vendedores mediante sistemas de reseñas y calificaciones, y al mejorar la experiencia de compra a través de flujos sencillos y eficaces. La solución no estaría restringida a una única ciudad gracias a su arquitectura escalable, sino que podría evolucionar de manera gradual para ajustarse a las dinámicas comerciales de cualquier comunidad urbana en el país.

Objetivos del proyecto:

Desarrollar una plataforma digital completa llamada Vecino, que tiene su fundamento en el modelo de mercado virtual hiperlocal. Esta plataforma posibilitará la conexión eficaz entre emprendedores, consumidores y comerciantes que se encuentren en la misma área geográfica, a través de un ecosistema tecnológico sólido que simplifique el manejo total del comercio local, desde la publicación de productos y el procesamiento de los pedidos hasta el análisis del rendimiento comercial. De esta manera, se podrá contribuir al desarrollo económico local en las ciudades colombianas, con la posibilidad de expandirse gradualmente a otras áreas urbanas del país.

Para alcanzar este propósito, el proyecto se estructura en torno a los siguientes objetivos específicos:

- Crear un sistema de registro de usuarios y negocios que permita a los comerciantes configurar su perfil y publicar productos o servicios de manera sencilla e intuitiva, reduciendo las barreras de entrada a la digitalización.
- Desarrollar un módulo de catálogo y carrito de compras que facilite a los usuarios explorar la oferta disponible, seleccionar productos y gestionar sus pedidos de forma ágil y sin fricciones.
- Diseñar una interfaz web adaptable a dispositivos móviles y de escritorio que garantice una experiencia de uso accesible, coherente y funcional independientemente del dispositivo desde el que se acceda a la plataforma.
- Construir un proceso de compra integral que incorpore la integración con una pasarela de pagos en línea, el procesamiento de pedidos con estados definidos y el carrito de compras, asegurando que las transacciones sean trazables y seguras.

- Garantizar un módulo de geolocalización que posibilite a los usuarios ver en un mapa interactivo las empresas cercanas a su localización, lo cual simplifica el hallazgo de la oferta local.

Alcance del proyecto:

La plataforma Vecino es una solución de software completa, enfocada en cambiar el funcionamiento del comercio local en las áreas urbanas de Colombia. Esta conecta a los comerciantes, emprendedores y clientes dentro de una misma área geográfica por medio de un ecosistema digital sólido, que puede crecer y tiene un gran impacto social. El sistema no se restringe a una función básica, sino que abarca un conjunto integral de módulos operativos que abarcan todo el ciclo de la experiencia comercial: desde el registro y la configuración del negocio, hasta la administración del catálogo, el procesamiento de pedidos, la comunicación en tiempo real entre las partes involucradas y la creación de informes y métricas para los comerciantes.

Alcance funcional del sistema:

el sistema Vecino estará compuesto por los siguientes módulos funcionales:

Tabla 1: *modulo funcional del sistema “Vecino”*

Modulo	Descripción general
Gestión de usuarios	Recuperación de la contraseña, registro, autenticación y administración de perfiles según el rol (administrador, comerciante o consumidor).
Gestión de negocios	Elaboración, modificación y gestión integral del perfil empresarial, que abarca la categoría, localización geográfica, horario de atención e imágenes.
Catálogo de productos	Lanzamiento, edición y distribución de productos por categorías, con imágenes, precios, inventario y disponibilidad.
Motor de búsqueda y filtros	Búsqueda de productos y empresas según su nombre, categoría, localización y calificación, con los resultados organizados por cercanía y relevancia.

Carrito de compras	Administración flexible del carrito, incluyendo la renovación de cantidades, el cálculo automático de totales y la verificación en tiempo real del inventario.
Procesamiento de pedidos	Registro, monitoreo y administración del ciclo total del pedido con estados establecidos: pendiente, confirmado, en preparación, en tránsito y entregado.
Pasarela de pagos	Integración con una pasarela de pagos en línea que haga posible realizar operaciones seguras desde la plataforma.
Sistema de calificaciones y reseñas	Módulo que permite a los clientes evaluar productos y negocios después de que han realizado una compra, mostrando el promedio en el perfil del negocio.
Módulo de geolocalización	De acuerdo con la localización del usuario, se muestra en un mapa interactivo una representación visual de los negocios que están cerca.
Administración del sistema	Panel administrativo para la administración de usuarios, la moderación de opiniones, los negocios y la configuración general del sistema.
Panel de administración del comerciante	Tablero de control con estadísticas de rendimiento del negocio, historial de pedidos, métricas de ventas y productos más comprados.

Aunque el sistema Vecino incluye una amplia gama de funcionalidades, hay elementos que no se desarrollan en este proyecto debido a su complejidad o alcance:

Tabla 2: aspectos que no estarán en el proyecto “Vecino”

Aspecto	Justificación
Aplicación móvil nativa (iOS / Android)	El sistema funciona en una plataforma web adaptativa. La creación de aplicaciones nativas es una etapa que sucede después del desarrollo del producto.

Sistema logístico propio de domicilios	La gestión de las entregas se realiza entre el comerciante y el cliente. La incorporación de operadores logísticos externos es un paso a futuro.
Inteligencia artificial y recomendaciones personalizadas	Las características de recomendación que se basan en el comportamiento del usuario están previstas como mejoras para versiones futuras.
Integración con sistemas contables externos	El alcance presente del proyecto no incluye la vinculación con plataformas como SIIGO o Alegra.
Expansión multiciudad automatizada	La plataforma se elabora con una estructura escalable para varias ciudades, sin embargo, en esta versión, la configuración y el despliegue se llevarán a cabo de forma manual en cada ciudad.

Beneficios esperados del sistema:

La implementación de la plataforma Vecino generará los siguientes beneficios para los actores involucrados:

Tabla 3: *beneficios esperados del sistema*

Beneficiario	Beneficio esperado
Comerciantes locales	Acceso a un canal digital propio sin costos elevados de implementación, mayor visibilidad en su comunidad y recursos para administrar su negocio.
Consumidores	Paso centralizado a la oferta local, posibilidad de respaldar el comercio en su área cercana y experiencia de compra rápida y fiable.
Comunidad local	Potenciación del consumo local, descenso de la dependencia de plataformas externas y afianzamiento del tejido económico barrial.

Equipo de desarrollo	Desarrollo práctico de habilidades en arquitectura de sistemas, desarrollo full stack, gestión de proyectos ágiles e ingeniería de software.

Metodología de desarrollo:

El proyecto se desarrollará utilizando la metodología ágil Scrum, un marco de trabajo iterativo e incremental que es ampliamente utilizado en el sector del software para manejar proyectos difíciles en contextos variables. Scrum organiza el trabajo en ciclos breves llamados Sprints, que tienen una duración estable de dos semanas. Al final de cada uno de estos períodos, se logra un aumento de la funcionalidad del producto. Este método posibilita que el equipo planifique de forma gradual, detecte obstáculos de manera temprana y modifique el desarrollo en función de las prioridades del proyecto.

La elección de Scrum como metodología está determinada por las particularidades del proyecto: un equipo pequeño, un ámbito limitado y la necesidad de proporcionar valor ininterrumpidamente durante el semestre académico.

Roles del equipo:

Scrum establece 4 roles esenciales que organizan la manera en que el equipo coopera y decide. El Product Owner es el encargado de determinar y establecer prioridades para las necesidades del producto, al tiempo que representa los intereses de los usuarios y las partes interesadas ante el equipo de desarrollo. El Scrum Máster se desempeña como facilitador del proceso, garantizando que el equipo entienda y utilice adecuadamente los principios de Scrum, eliminando obstáculos que puedan interferir con el progreso del proyecto. Por último, el equipo de desarrollo está compuesto por los miembros que tienen la responsabilidad de diseñar, construir y entregar las mejoras funcionales del producto en cada Sprint.

El equipo está formado por cuatro miembros, por lo que las funciones se organizarán de este modo:

Tabla 4: *roles del equipo*

Integrante	Rol
Santiago José Barbosa Rivas	Product Owner
Jerson Javier Ramírez Ricardo	Scrum Máster
Mario Alexander Avellaneda Buitrago	Desarrollador BackEnd
José Luis Arias	Desarrollador FrontEnd y testing

Organización del trabajo mediante Sprints:

El proyecto se desarrollará en cuatro Sprints, cada uno de ellos enfocado en la creación de un conjunto particular de funcionalidades del sistema. El siguiente es el plan general:

- Sprint 1: fundamentos y autenticación: se establece la estructura del proyecto, el entorno de desarrollo y el módulo para que los usuarios y negocios se registren e inicien sesión.
- Sprint 2: manejo de productos, roles y negocios: elaboración y modificación de perfiles comerciales, publicación de productos con un catálogo básico y fotografías, además con funciones de administrador, comerciante y cliente.
- Sprint 3: proceso de compra: módulo de carrito de compras, registro y monitoreo de pedidos por el usuario.
- Sprint 4: cierre y perfeccionamiento, que incluye la corrección de fallos, la geolocalización, reseñas y la adaptación de la usabilidad, las últimas pruebas y la integración de una interfaz responsiva.

Las ceremonias que incluirá cada Sprint son las siguientes: Sprint Planning al comienzo para determinar las actividades del ciclo, Daily Standup como reunión diaria de seguimiento, Sprint Review al término para examinar el incremento entregado y Sprint Retrospective para analizar el procedimiento y sugerir mejoras.

Herramientas de gestión:

Las siguientes herramientas se emplearán para la administración del proyecto:

- Figma y Stich para diseñar prototipos de alta y baja fidelidad.
- GitHub para controlar las versiones del código fuente, con repositorios separados para el backend y FrontEnd.
- Jira como plataforma principal para crear y seguir historias de usuario, administrar el backlog y supervisar el progreso en cada Sprint.
- Stack tecnológico establecida para la creación: Node.js con Express en la parte de BackEnd, React en la parte FrontEnd, PostgreSQL como sistema de almacenamiento y JWT para administrar la autenticación.

Figura 1: *tablero Jira, gestión de historias de usuario y sprints del proyecto Vecino*

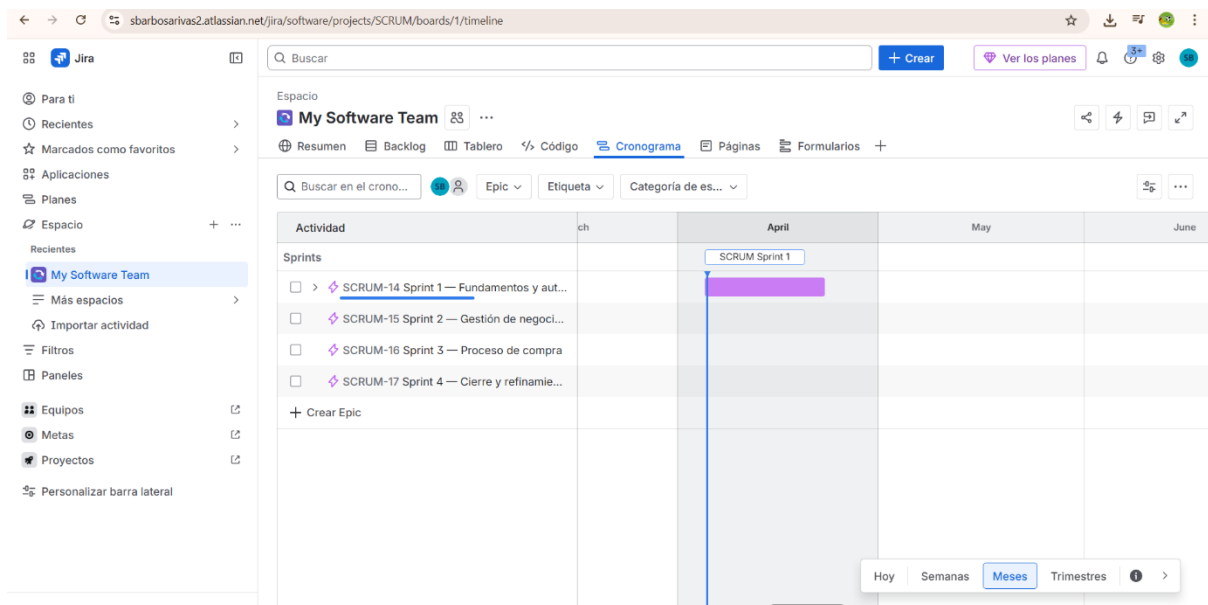


Figura 2: tablero de gestión del proyecto Vecino en Jira, Backlog con historias de usuario organizadas por Sprint

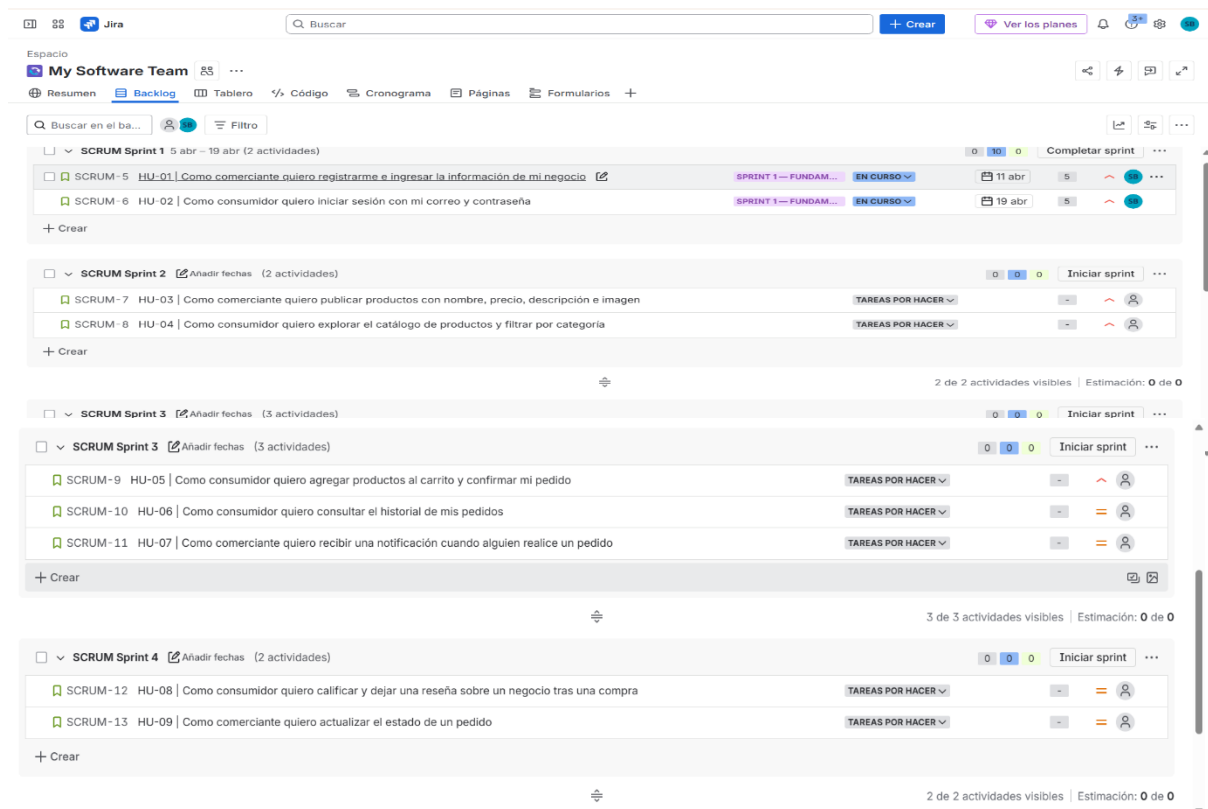


Figura 3: Git Hub del repositorio del BackEnd

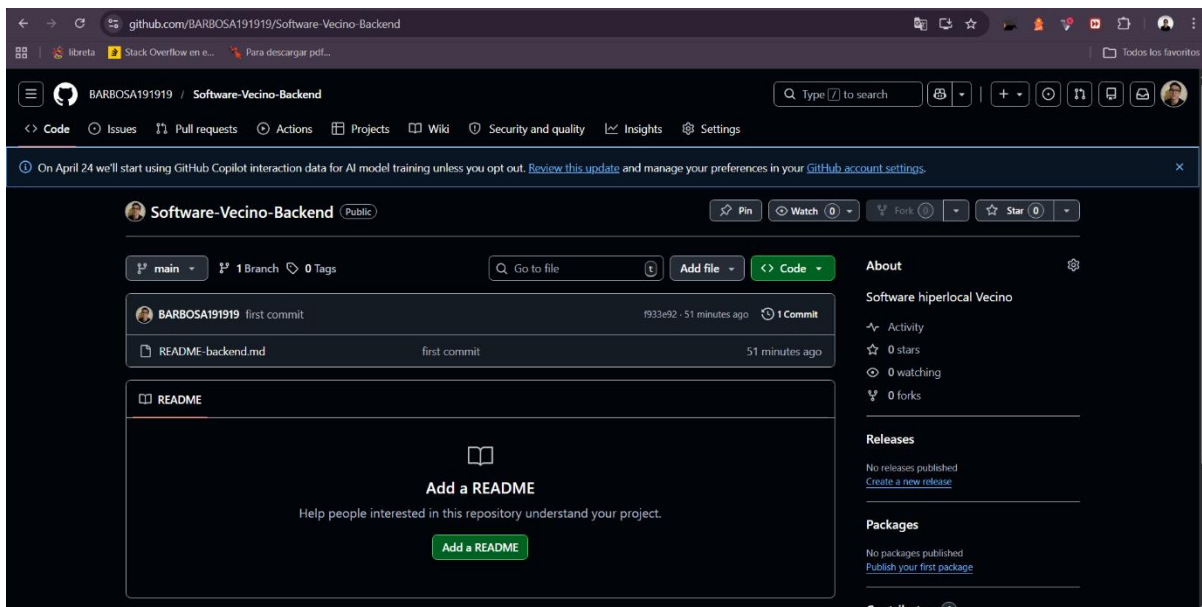
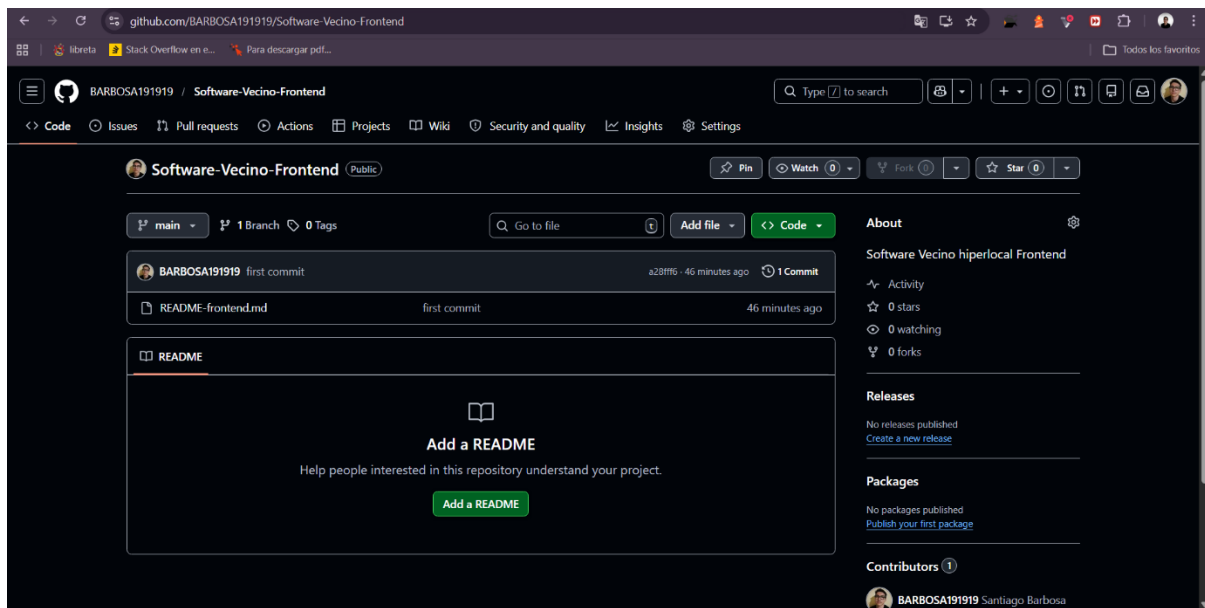


Figura 4: Git Hub del repositorio del FrontEnd



1. **Repositorio BackEnd Git Hub:**
<https://github.com/BARBOSA191919/Software-Vecino-Backend>
2. **Repositorio FrontEnd Git**
Hub:<https://github.com/BARBOSA191919/Software-Vecino-Frontend>
3. **Jira para planeación de historias de usuario y desarrollo:**
<https://sbarbosarivas2.atlassian.net/jira/software/projects/SCRUM/boards/1>

Requisitos del sistema:

En el desarrollo de software, una fase esencial es la identificación y documentación de los requisitos del sistema, pues define con exactitud las acciones que debe realizar el sistema y las condiciones de calidad bajo las cuales debe funcionar. De acuerdo con el IEEE (Instituto de Ingenieros Eléctricos y Electrónicos, 2011), un requisito es una capacidad o condición que un sistema debe tener para poder cumplir con las necesidades de sus usuarios. Para el presente proyecto se distinguen dos categorías: requisitos funcionales, que describen las funciones y servicios que el sistema debe proveer, y requisitos no funcionales, que definen los atributos de calidad que debe cumplir, alineados con el estándar internacional ISO/IEC 25010 (2011).

Tabla 5: requisitos funcionales

ID	Requisito	Descripción	Prioridad
RF-01	Registro de usuarios	El sistema tiene que posibilitar que los usuarios se registren como comerciantes o consumidores, pidiendo para ello nombre, correo electrónico y contraseña.	Alta
RF-02	Autenticación	El sistema tiene que validar a los usuarios a través de correo y contraseña, creando un token JWT con tiempo de vencimiento.	Alta
RF-03	Gestión de perfil de negocio	El comerciante debe tener la posibilidad de crear, modificar y borrar el perfil de su negocio, lo que incluye la imagen, el nombre, la categoría y la descripción.	Alta
RF-04	Publicación de productos	El vendedor tiene que ser capaz de publicar, modificar y borrar productos con stock, nombre, precio, imagen y descripción.	Alta
RF-05	Exploración del catálogo	El cliente debe tener la posibilidad de examinar los productos que están disponibles y filtrarlos según su categoría.	Alta
RF-06	Carrito de compras	Antes de confirmar el pedido, el sistema tiene que darle la opción al cliente de añadir, quitar o cambiar productos del carrito.	Alta
RF-07	Registro de pedidos	El sistema debe registrar las órdenes que hace el cliente y notificar al vendedor correspondiente.	Alta
RF-08	Historial de pedidos	El comerciante y el cliente deben tener la capacidad de revisar el historial de los pedidos que han hecho y recibido, respectivamente.	Media
RF-09	Calificaciones y reseñas	El cliente tiene que tener la posibilidad de evaluar una empresa y dejar un comentario después de efectuar un pedido.	Media
RF-10	Gestión de estados de pedido	El vendedor tiene que ser capaz de modificar el estado del pedido a: pendiente, en preparación o entregado.	Media

Los requisitos no funcionales del sistema se determinan en función de las propiedades de calidad definidas por la norma ISO/IEC 25010 (2011), que ofrece un modelo de calidad del producto de software ampliamente utilizado en el sector. Este estándar categoriza la calidad del software en ocho atributos clave, de los cuales el proyecto actual se enfoca en los siguientes:

Tabla 6: requisitos no funcionales

ID	Característica ISO 25010	Requisito	Descripción
RF-01	Seguridad	Autenticación con JWT	Es necesario manejar todas las sesiones con tokens JWT que tengan una duración establecida. La comunicación entre el cliente y el servidor tiene que llevarse a cabo usando HTTPS.
RF-02	Seguridad	Protección de datos	Las contraseñas tienen que guardarse encriptadas a través de algoritmos de hashing, como el bcrypt.
RF-03	Eficiencia de desempeño	Tiempo de respuesta	En circunstancias normales de uso, el sistema tiene que responder a las peticiones del usuario en menos de 3 segundos.
RF-04	Fiabilidad	Disponibilidad	Durante las horas de funcionamiento, la plataforma tiene que sostener una disponibilidad mínima del 95%.
RF-05	Usabilidad	Interfaz responsiva	La interfaz tiene que ajustarse de manera adecuada a dispositivos móviles y de escritorio sin que se pierda la funcionalidad.
RF-06	Usabilidad	Facilidad de uso	Para el usuario, el proceso de registro y su primera compra no debería tomar más de cinco pasos.
RF-07	Mantenibilidad	Estructura modular	Para que el mantenimiento y la escalabilidad sean más fáciles, el código debe estructurarse en módulos autónomos por dominio (pedidos, productos, negocios y usuarios).
RF-08	Portabilidad	Compatibilidad de navegadores	La plataforma tiene que operar adecuadamente en las versiones más

			actualizadas de los navegadores Chrome, Edge, Firefox y Safari.
RF-09	Escalabilidad	Crecimiento de usuarios	Para poder soportar un número más elevado de usuarios sin tener que rediseñar la estructura, es necesario que la arquitectura facilite el escalado horizontal de los servicios del backend.

Identificación de stakeholders y usuarios finales:

Cada proyecto de software incluye participantes con diversos grados de interés y participación en el desarrollo y los resultados del sistema. Según el Project Management Institute (PMI, 2021), los stakeholders son personas, grupos o entidades que tienen la capacidad de influir en el proyecto o ser influenciados por él. Para el sistema Vecino, se determinan los siguientes:

Tabla 7: *identificación de stakeholders y usuarios finales*

Actor	Tipo	Rol en el proyecto	Interés principal
Comerciantes locales	Usuario final	Publicar productos, gestionar pedidos y ofrecer sus servicios dentro de la plataforma	Aumentar sus ventas y tener mayor visibilidad en su comunidad
Consumidores (usuarios)	Usuario final	Buscar productos, realizar compras y calificar negocios	Encontrar productos cercanos de forma rápida, económica y confiable
Administrador del sistema	Stakeholder interno	Gestionar la plataforma, controlar usuarios, supervisar contenido y funcionamiento	Garantizar el correcto funcionamiento del sistema y la calidad del servicio

Desarrollador(es) del sistema	Stakeholder interno	Diseñar, desarrollar, implementar y mantener la plataforma	Crear una solución funcional, estable y escalable
Inversionistas o patrocinadores	Stakeholder externo	Financiar o apoyar el desarrollo del proyecto	Obtener retorno de inversión o impacto económico/social
Comunidad local	Stakeholder externo	Beneficiarse indirectamente del uso de la plataforma	Fortalecimiento de la economía local y generación de oportunidades
Repartidores independientes (opcional)	Usuario indirecto	Apoyar la entrega de pedidos (si aplica en el futuro)	Generar ingresos mediante servicios de entrega

Historias de usuario:

Las historias de usuario son un método característico de las metodologías ágiles que posibilita la descripción de las funcionalidades del sistema desde el punto de vista del usuario final. Cohn (2004) sostiene que una historia de usuario es un relato breve y sencillo de una funcionalidad, contada desde el punto de vista del usuario que la necesita. Es decir que las historias de usuario, en el contexto de Scrum, forman parte del Product Backlog y funcionan para la planificación de cada Sprint. Se exponen a continuación las historias de usuario que se establecieron para la versión inicial del sistema Vecino:

Tabla 8: *historia de usuario del sistema Vecino*

ID	Rol	Quiero	Para	Criterios de aceptación	Prioridad	Sprint
HU-01	Comerciante	Inscribirme en la plataforma e introducir los datos de mi empresa	Estar presente en el entorno digital y empezar a brindar mis productos	El sistema autentica los datos obligatorios, genera el perfil y manda una confirmación.	Alta	Sprint 1
HU-02	Consumidor	Iniciar sesión con mi correo y contraseña	Entrar de manera segura a mi cuenta	El sistema autentica mediante JWT y redirige al comienzo.	Alta	Sprint 1
HU-03	Comerciante	Publicar productos con nombre, precio, descripción e imagen	Que los clientes tengan conocimiento de mi oferta y sean capaces de adquirirla.	El producto se muestra en el catálogo y es perceptible para los clientes.	Alta	Sprint 2
HU-04	Consumidor	Explorar el catálogo de productos y filtrar por categoría	Ubicar con facilidad lo que requiero cerca de donde me encuentro.	Los artículos se enumeran de manera adecuada y el filtro opera según la categoría.	Alta	Sprint 2
HU-05	Consumidor	Agregar productos al carrito y confirmar mi pedido	Administrar mis compras sin tener que hablar directamente con el vendedor	El pedido es registrado y, a continuación, se notifica al comerciante.	Alta	Sprint 3

HU-06	Consumidor	Consultar el historial de mis pedidos	Monitorear mis compras y llevar un registro de mis operaciones	El historial exhibe la fecha, el estado y el detalle de cada orden.	Media	Sprint 3
HU-07	Comerciante	Recibir una notificación cuando alguien realice un pedido	Responder pronto y atenderlo a tiempo	El sistema avisa al comerciante en tiempo real o cuando se recarga.	Media	Sprint 3
HU-08	Consumidor	Calificar y dejar una reseña sobre un negocio tras realizar una compra	Ayudar a otros usuarios y transmitir mi experiencia	La reseña está vinculada con el negocio y se muestra en su perfil.	Media	Sprint 4
HU-09	Comerciante	Actualizar el estado de un pedido	Mantener al cliente informado acerca del progreso de su adquisición	El historial del consumidor le permite ver el estado actualizado.	Media	Sprint 5

Flujos de Solución

Figura 5: *flujo general del sistema Vecino según rol del usuario*

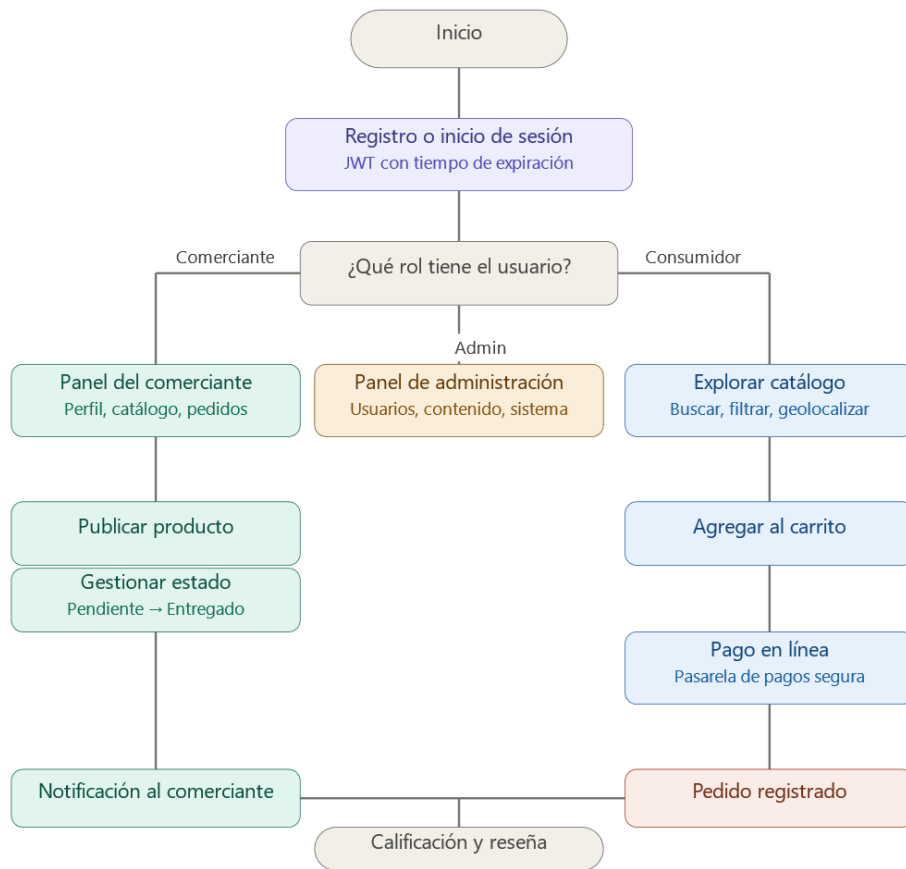


Figura 6: *proceso de compra del consumidor*

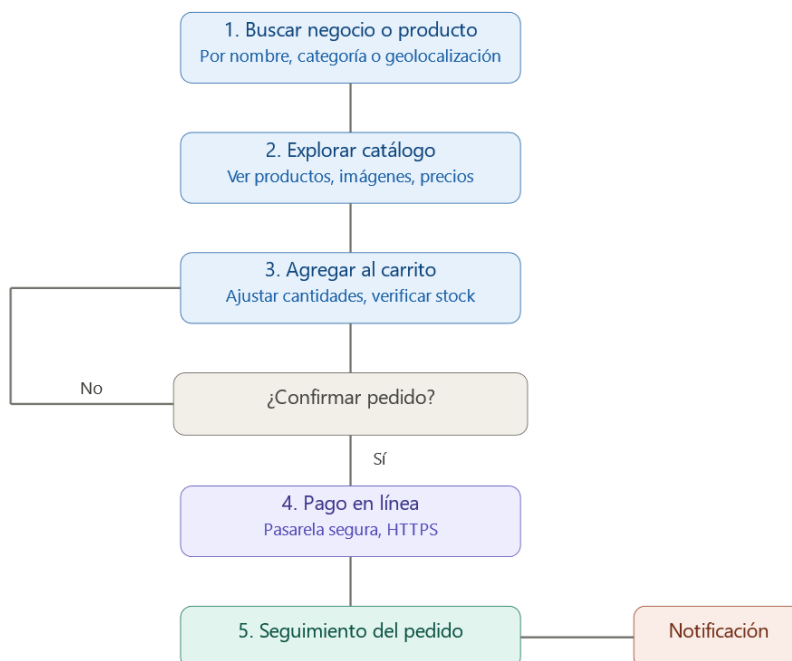


Figura 7: interacción del comerciante con el sistema

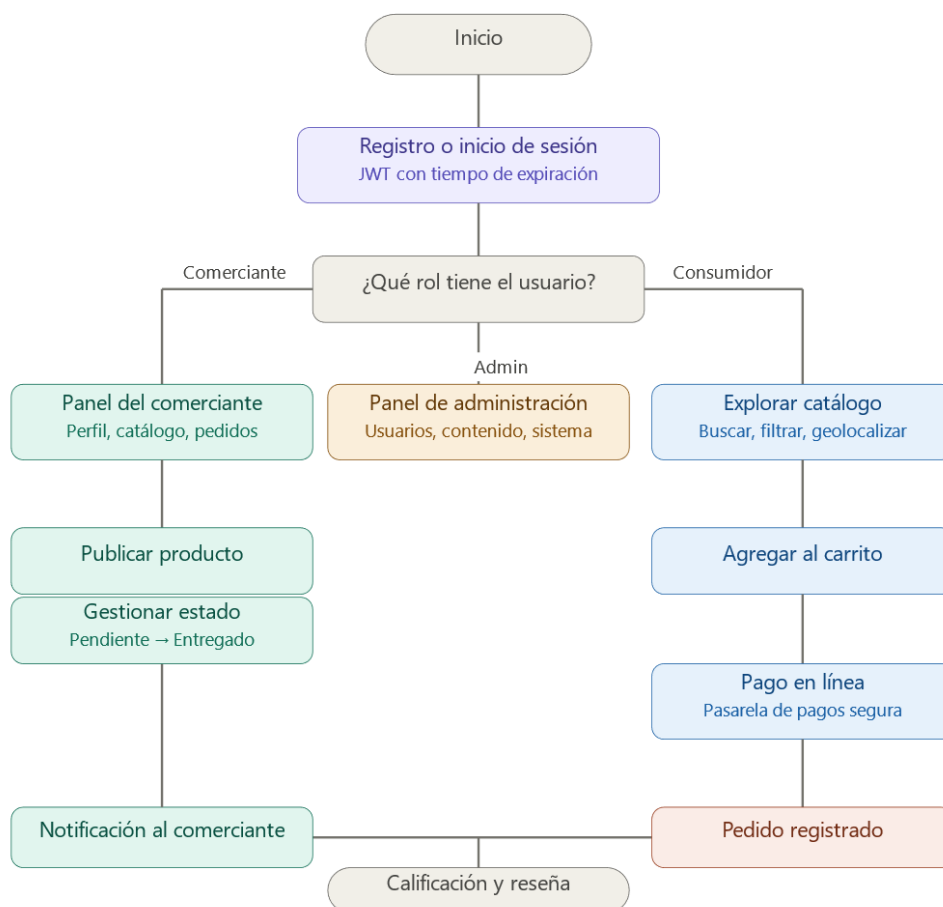


Figura 8: diagrama de casos de uso del sistema Vecino

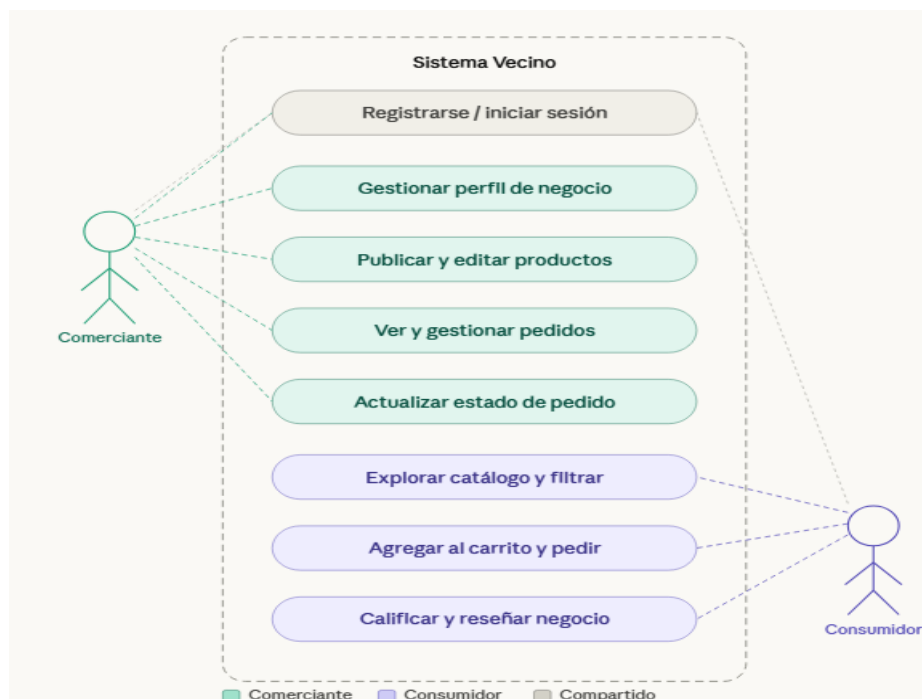


Figura 9: diagrama entidad-relación del sistema Vecino

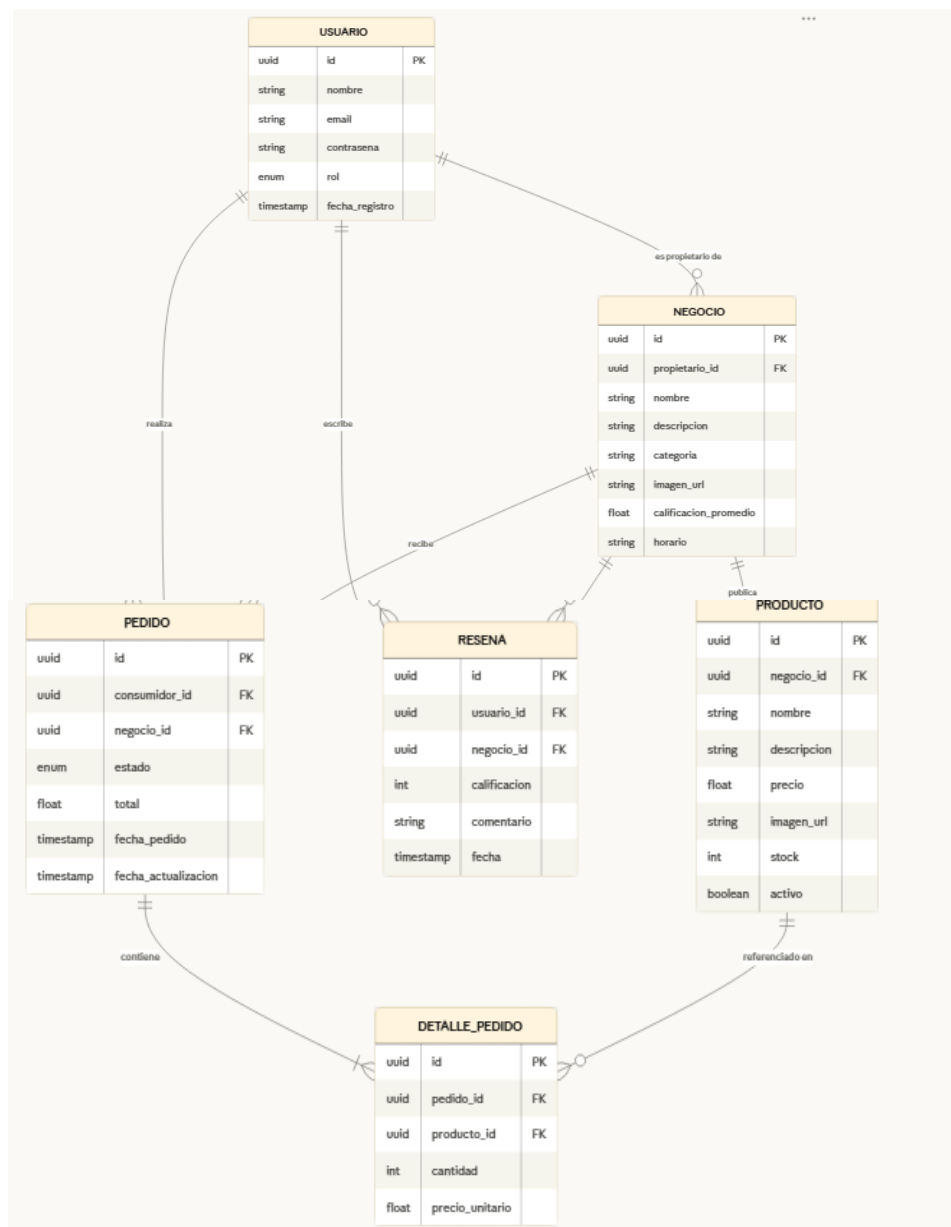
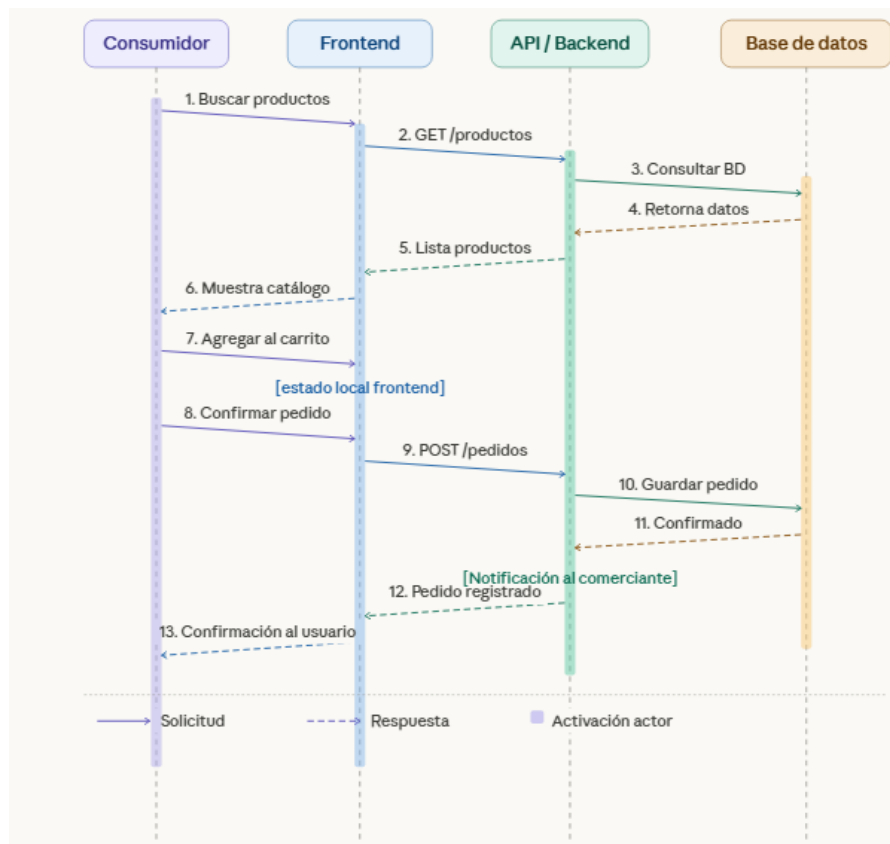


Figura 10: *diagrama de secuencia, flujo de compra del consumidor en el sistema Vecino*



Solución tecnológica a implementar

Descripción del sistema:

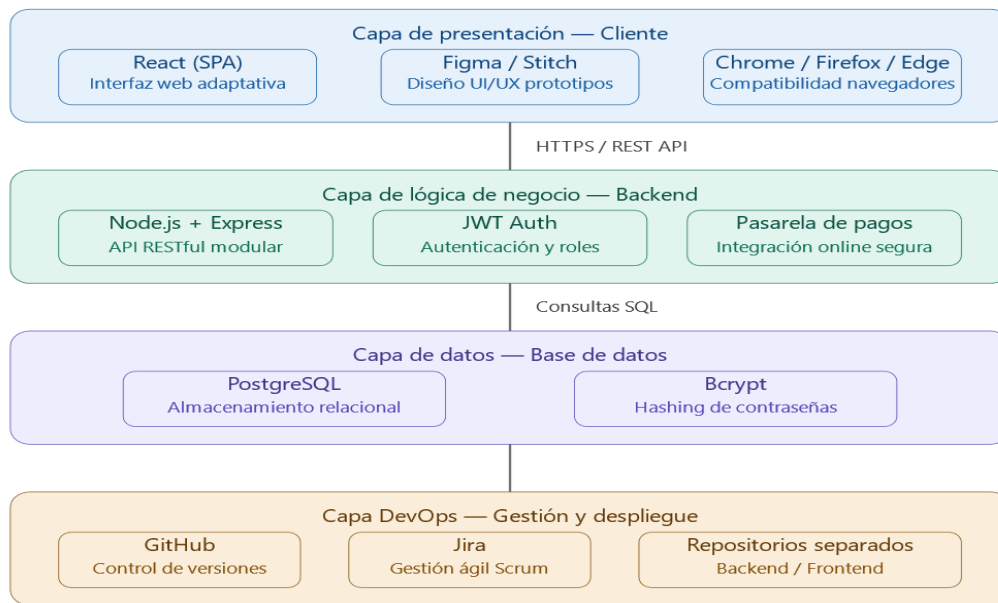
Vecino es una plataforma digital de tipo marketplace hiperlocal orientada a conectar comerciantes, emprendedores y consumidores dentro de una misma área geográfica en ciudades intermedias de Colombia. El sistema permite a los comerciantes locales crear su perfil de negocio, publicar productos y gestionar pedidos de forma autónoma, mientras que los consumidores pueden explorar la oferta cercana, realizar compras en línea y calificar los negocios con los que interactúan.

La plataforma opera como una solución web adaptativa, accesible desde dispositivos móviles y de escritorio, sin requerir la instalación de una aplicación nativa. Su propósito central es reducir la brecha digital del comercio local, ofreciendo una herramienta tecnológica asequible que fortalezca el tejido económico barrial y disminuya la dependencia de plataformas externas de gran escala.

El sistema está compuesto por once módulos funcionales que cubren el ciclo comercial completo: desde el registro y autenticación de usuarios, pasando por la gestión de negocios y catálogos, el procesamiento de pedidos con estados definidos, la integración con una pasarela de pagos en línea, hasta el módulo de geolocalización y el panel de métricas para comerciantes.

Arquitectura general de la solución:

La arquitectura de Vecino sigue un modelo cliente-servidor en tres capas, con separación clara entre el frontend, el backend y la base de datos.

Figura 11: *arquitectura general de la solución Vecino*

Por consiguiente, la arquitectura se organiza en cuatro capas. La capa de presentación es responsable de toda la interacción con el usuario a través de una SPA construida en React, garantizando una interfaz responsiva compatible con los principales navegadores. La capa de lógica de negocio expone una API RESTful construida con Node.js y Express, que centraliza la autenticación mediante JWT, el procesamiento de pedidos, y la integración con la pasarela de pagos. La capa de datos utiliza PostgreSQL como sistema de gestión relacional, con contraseñas protegidas mediante Bcrypt. Finalmente, la capa DevOps gestiona el control de versiones con GitHub y el seguimiento del proyecto con Jira bajo la metodología Scrum.

Tecnologías que se utilizarán:

Tabla 9: *tecnologías que se utilizaran en el sistema Vecino*

Área	Tecnología	Propósito
Frontend	React	Construcción de la interfaz de usuario como SPA
Backend	Node.js + Express	Servidor y API RESTful
Base de datos	PostgreSQL	Almacenamiento relacional de datos
Autenticación	JWT	Gestión de sesiones y roles de usuario

Seguridad	Bcrypt	Encriptación de contraseñas
Diseño	Figma + Stitch	Prototipado en baja y alta fidelidad
Versionamiento	GitHub	Control de código fuente (repos separados)
Gestión ágil	Jira	Backlog, sprints e historias de usuario
Comunicación	HTTPS	Transmisión segura de datos
Pagos	Pasarela en línea	Procesamiento seguro de transacciones

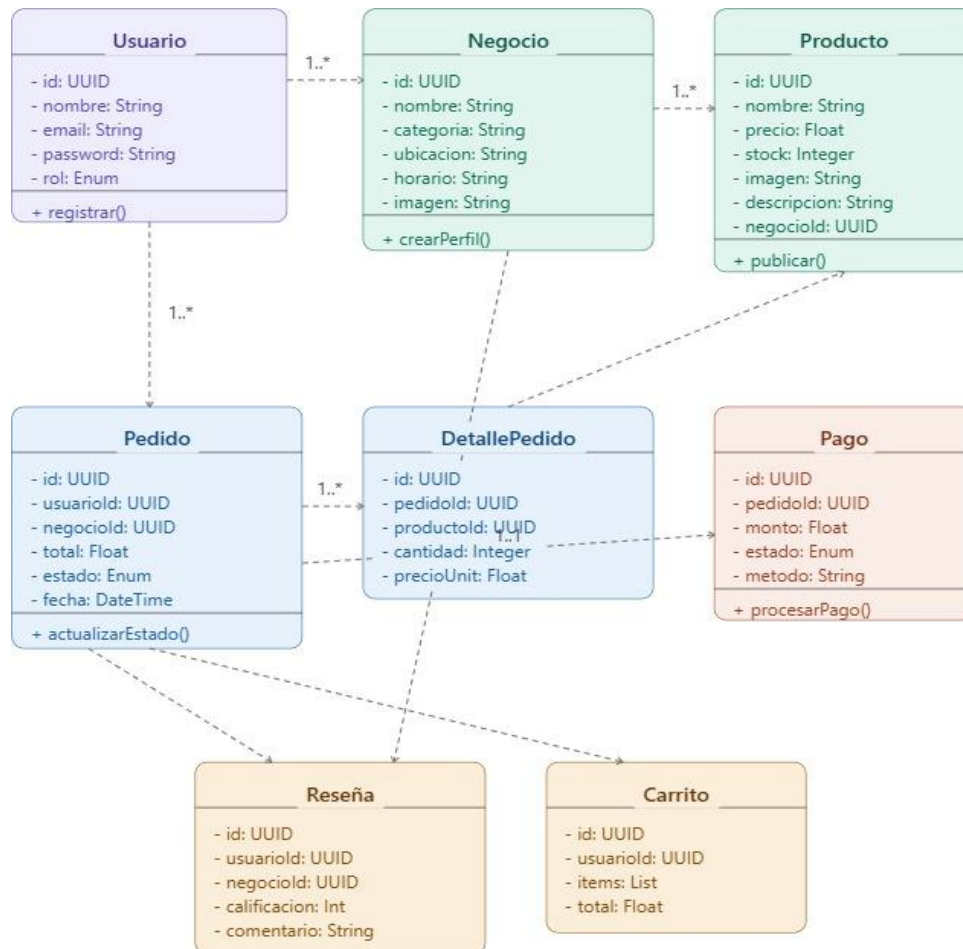
Beneficios esperados del sistema:

De acuerdo con lo establecido en el documento (Tabla 3), los beneficios se distribuyen entre los distintos actores involucrados:

1. **Para los comerciantes locales**, la plataforma representa un canal de venta digital propio sin los altos costos de implementación de plataformas tradicionales. Tendrán mayor visibilidad dentro de su comunidad y acceso a un panel de métricas con historial de pedidos y estadísticas de rendimiento que les permitirá tomar decisiones informadas sobre su negocio.
2. **Para los consumidores**, Vecino ofrece un punto de acceso centralizado a la oferta local, con la posibilidad de encontrar productos y servicios cercanos de forma rápida, confiable y económica. El sistema de calificaciones y reseñas genera confianza en cada transacción, y la geolocalización simplifica el descubrimiento de negocios en su entorno inmediato.
3. **Para la comunidad local**, la plataforma potencia el consumo dentro del mismo territorio, reduce la dependencia de aplicaciones externas y fortalece el tejido económico barrial al mantener la circulación del dinero dentro de la misma comunidad.
4. **Para el equipo de desarrollo**, el proyecto representa la aplicación práctica de habilidades en arquitectura de sistemas, desarrollo full stack con el stack Node.js + Express + React + PostgreSQL, gestión de proyectos ágiles con Scrum, e ingeniería de software en un contexto real y contextualizado.

Modelamiento del sistema

Figura 12: *diagrama de clases del sistema Vecino*



El sistema cuenta con ocho clases principales: Usuario gestiona los tres roles del sistema (administrador, comerciante, consumidor); Negocio y Producto pertenecen al dominio del comerciante; Pedido y DetallePedido modelan el proceso de compra; Pago integra la pasarela en línea; y Reseña y Carrito complementan la experiencia del consumidor. *Los tipos de datos reflejan la implementación real en PostgreSQL. UUID corresponde al tipo nativo de identificador único universal soportado por el motor de base de datos.*

Figura 13: prototipo de alta fidelidad, registro de usuario

The image shows a high-fidelity prototype of a user registration screen for a platform called 'Vecino'. The header includes the 'Vecino' logo, a progress indicator for 'PASO 1 DE 2', and links for '¿Ya tienes cuenta?' and 'Inicia sesión'. The main heading is 'Únete a la plaza digital' with a subtext: 'Selecciona cómo deseas participar hoy en Vecino. Podrás cambiar esto más tarde.' Below this are two large buttons: 'Quiero comprar' (with a shopping bag icon) and 'Quiero vender' (with a storefront icon). Each button has a brief description of the user's role. Below these is a form titled 'Información básica' with fields for 'Nombre completo' (Example: Ana García), 'Correo electrónico' (Example: ana@vecino.com), and 'Contraseña' (with a strength indicator and a 'Mostrar/Ocultar' toggle). A 'Continuar registro' button is at the bottom, followed by a small link to 'Algunas reglas de nuestra Política de Servicio y Política de Privacidad'.

Nota. Prototipo interactivo de alta fidelidad correspondiente al módulo de registro.

Elaboración propia (2026). Disponible en:

<https://stitch.withgoogle.com/preview/6944331621580144500?node-id=5029de78e95a4fff9cc618a5de60b19b&raw=1>

Figura 14: prototipo de alta fidelidad, recuperación de contraseña

The image shows a high-fidelity prototype of a password recovery screen for the 'Vecino' platform. The header features the 'Vecino' logo. The main heading is 'Forgot password?' with a subtext: 'Enter your email to reset your password. We'll send a link to get you back into your neighborhood square.' Below this is an 'Email address' input field with the placeholder 'neighbor@example.com'. A 'Send reset link' button is positioned below the input field. At the bottom of the form is a 'Back to Login' link. The footer includes the 'Vecino' logo, links for 'Privacy Policy', 'Terms of Service', and 'Help Center', and a copyright notice: '© 2024 Vecino. Built for the community.'

Nota. Prototipo interactivo de alta fidelidad correspondiente al módulo de

recuperación de contraseña. Elaboración propia (2026). Disponible en:

<https://stitch.withgoogle.com/preview/6944331621580144500?node-id=9a1d9805b4694778b56b23ee7779143a>

Figura 15: *prototipo de alta fidelidad, inicio de sesión*

Nota. Prototipo interactivo de alta fidelidad correspondiente al módulo de inicio de sesión. Elaboración propia (2026). Disponible en:

<https://stitch.withgoogle.com/preview/6944331621580144500?node-id=3b6d59f5ce58437c9a610d6a59edf7fb>

Figura 16: *prototipo de alta fidelidad, gestión de negocios*

Día	Horario
Lunes	07:00 AM - 08:00 PM
Martes	07:00 AM - 08:00 PM
Miércoles	07:00 AM - 08:00 PM
Jueves	CERRADO
Viernes	07:00 AM - 08:00 PM
Sábado	08:00 AM - 10:00 PM
Domingo	08:00 AM - 02:00 PM

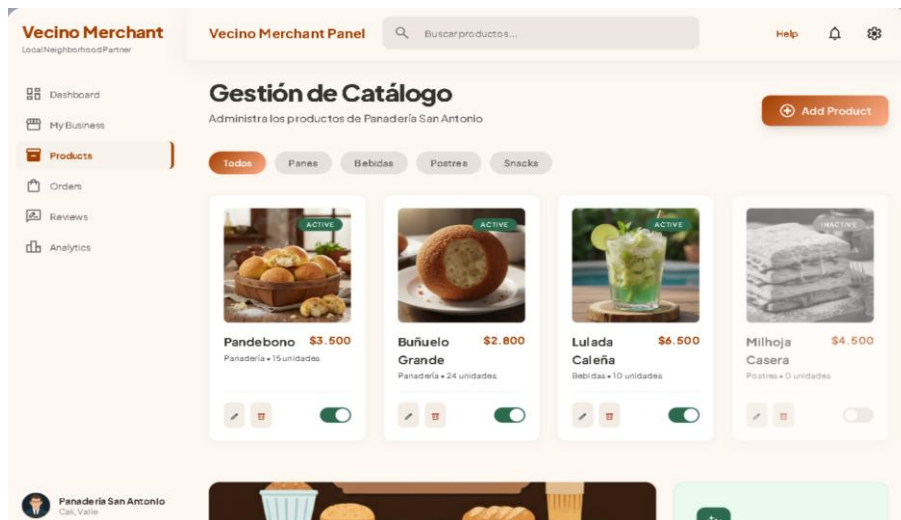
Nota.

de alta fidelidad correspondiente al módulo de gestión de negocios. Elaboración propia (2026). Disponible en:

<https://stitch.withgoogle.com/preview/6944331621580144500?node-id=3351b102dcc142ce93e0ac6e34a9cefa>

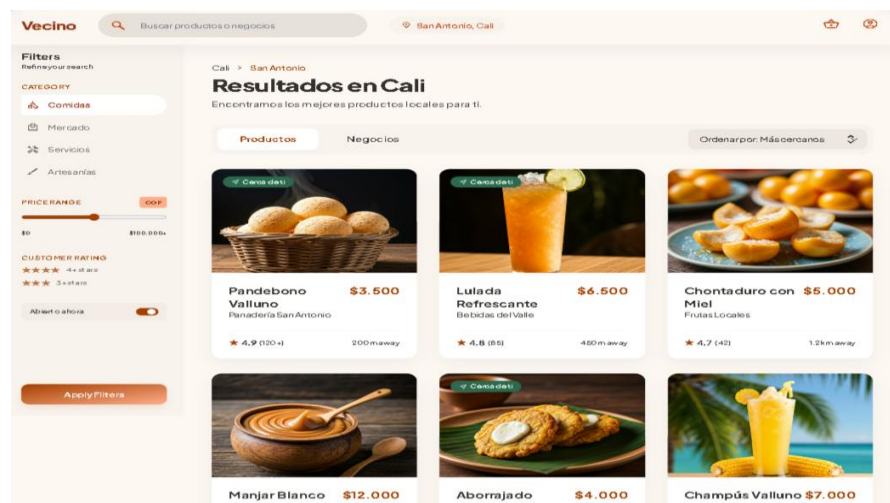
Prototipo interactivo

Figura 17: prototipo de alta fidelidad, catálogo de productos



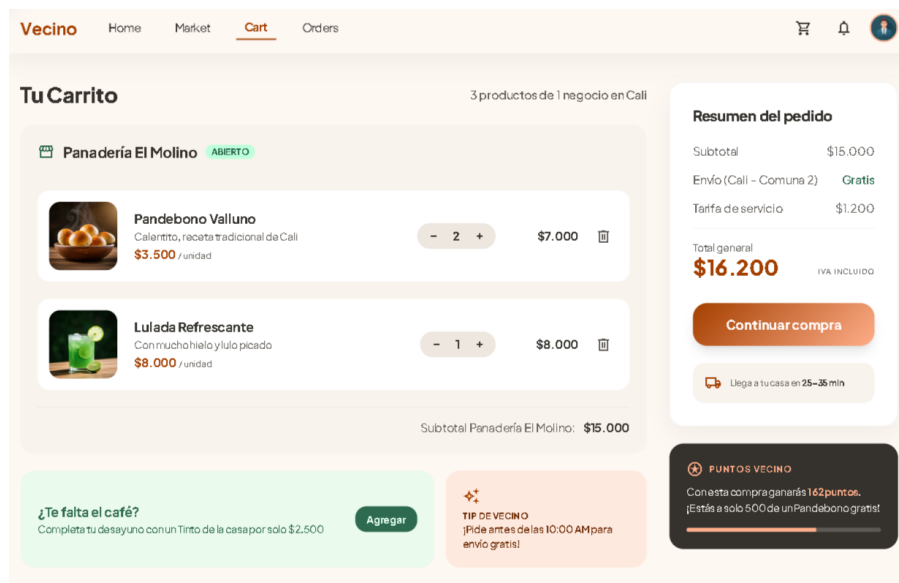
Nota. Prototipo interactivo de alta fidelidad correspondiente al módulo de alta fidelidad catálogo de productos. Elaboración propia (2026). Disponible en: <https://stitch.withgoogle.com/preview/6944331621580144500?node-id=ca2eb6c7b003427fa7705572356d0e49>

Figura 18: prototipo de alta fidelidad, motor de búsqueda y filtros



Nota. Prototipo interactivo de alta fidelidad correspondiente al módulo de alta fidelidad, motor de búsqueda y filtros. Elaboración propia (2026). Disponible en: <https://stitch.withgoogle.com/preview/6944331621580144500?node-id=1c0a2fed1ee1474981b21275b5b48c2d>

Figura 19: *prototipo de alta fidelidad, carrito de compras*

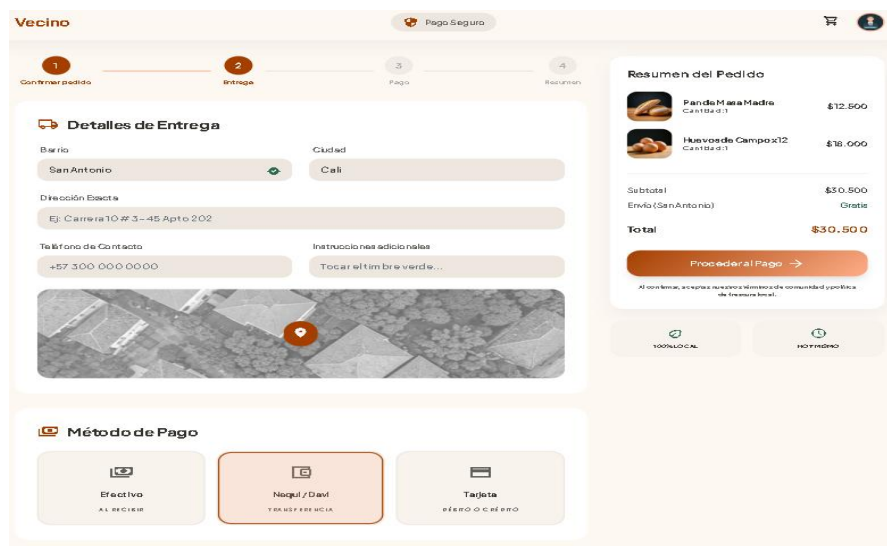


Nota.

Prototipo interactivo de alta fidelidad correspondiente al módulo carrito de compras. Elaboración propia (2026). Disponible en:

<https://stitch.withgoogle.com/preview/6944331621580144500?node-id=cd1a3b7b159943b18f4c323d8d3d2c49>

Figura 20: *prototipo de alta fidelidad, registro de pedido*

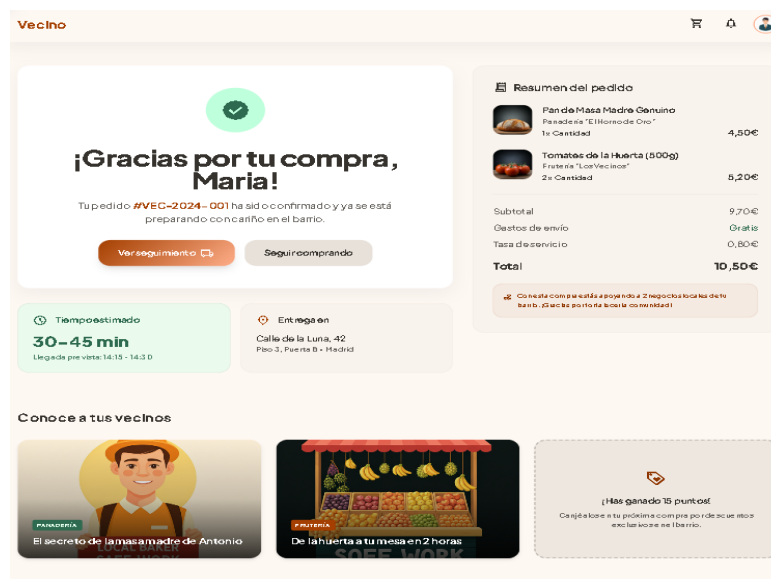


Nota.

Prototipo interactivo de alta fidelidad correspondiente al módulo registro de pedido. Elaboración propia (2026). Disponible en:

<https://stitch.withgoogle.com/preview/6944331621580144500?node-id=3665016fef804928a75840ad5d3d0b53>

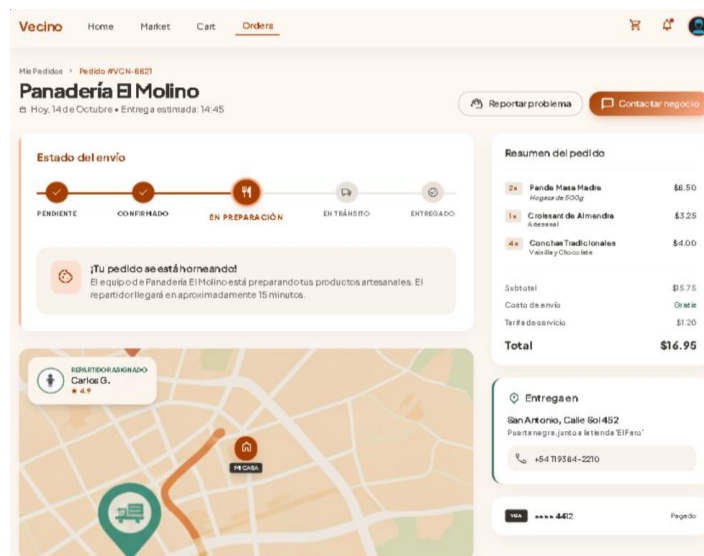
Figura 21: prototipo de alta fidelidad, confirmación de pedido



Nota.
interactivo
fidelidad correspondiente a la confirmación de pedido. Elaboración propia (2026).
Disponble en:

<https://stitch.withgoogle.com/preview/6944331621580144500?node-id=5d899728aff246e295c46157fba15713>

Figura 22: prototipo de alta fidelidad, seguimiento de pedido



Nota.
de alta fidelidad correspondiente al seguimiento de pedido. Elaboración propia
(2026). Disponible en:

<https://stitch.withgoogle.com/preview/6944331621580144500?node-id=eeb3668d48244b848803284c7de30dc4>

Figura 23: *prototipo de alta fidelidad, pasarela de pagos*

Vecino Pago Seguro

Finaliza tu Pago Seguro

Calli, Valle del Cauca • Panadería San Antonio

Método de Pago PASO 2 DE 2

Tarjeta de Crédito / Débito
Visa, Mastercard, American Express

NÚMERO DE LA TARJETA
0000 0000 0000 0000

VENCIMIENTO
MM/AA

CVC
123

PSE (Débito Bancario)
Paga directamente desde tu banco

Billeteras Digitales
Nequi, Daviplata

Resumen de tu Pedido
Panadería San Antonio • Calli

- Pan de bonos Vallunos (x6) Recién hornados \$18.000
- Lulada Tradicional (x2) Con hielo trappé y limón \$10.500

Subtotal \$28.500
Domicilio ¡GRATIS!

TOTAL A PAGAR \$28.500 COP IVA INCLUIDO

Pagar \$28.500 COP

Al hacer clic en "Pagar", aceptas nuestros [Términos de Servicio](#) y [Política de Privacidad](#). Tu transacción es 100% segura.

Entrega estimada: 25-35 min

Nota.

interactivo de alta fidelidad correspondiente a la pasarela de pagos. Elaboración propia (2026). Disponible en:

<https://stitch.withgoogle.com/preview/6944331621580144500?node-id=5d450a639b744aa19eae459a9c39afa2>

Figura 24: *prototipo de alta fidelidad, sistema de calificaciones*

VecinoMarket

¡Pedido Entregado!

Hola María, gracias por apoyar a Panadería San Antonio en Calli. Tu opinión ayuda a que nuestra comunidad crezca.

PRODUCTO
Pan de bonos Vallunos
\$3.500 COP c/u

¿Qué te pareció el producto?

★ ★ ★ ★ ★
Malo Regular Bueno Muy bueno ¡Excelente!

Califica el servicio de entrega

Deficiente Excelente

Comparte tu experiencia

¿Qué tal estaban los pan de bonos? Cuéntanos detalles...

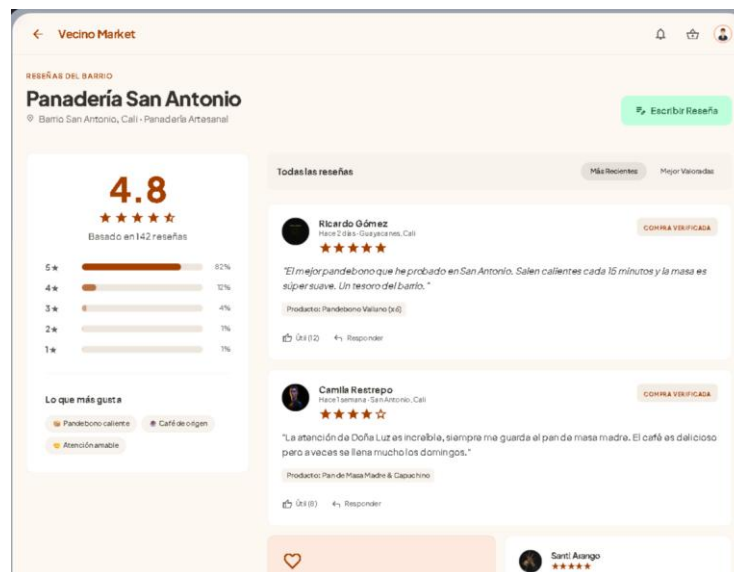
Tu reseña ayuda a que Panadería San Antonio sea más visible para otros vecinos en Calli ¡juntos fortalecemos el comercio local!

Enviar Reseña

Nota. *Prototipo interactivo de alta fidelidad correspondiente al sistema de calificaciones. Elaboración propia (2026). Disponible en:*

<https://stitch.withgoogle.com/preview/6944331621580144500?node-id=38abc243643d41a792f4f6d5b03daaf4>

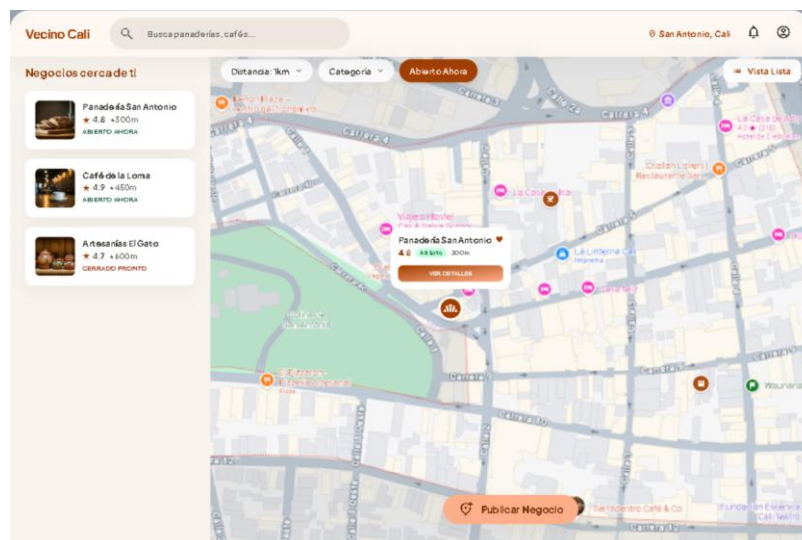
Figura 25: prototipo de alta fidelidad, sistema de reseñas



Nota. Prototipo interactivo de alta fidelidad correspondiente al sistema de reseñas. Elaboración propia (2026). Disponible en:

<https://stitch.withgoogle.com/preview/6944331621580144500?node-id=6933fa9772394162a9e77a37cc323dc9>

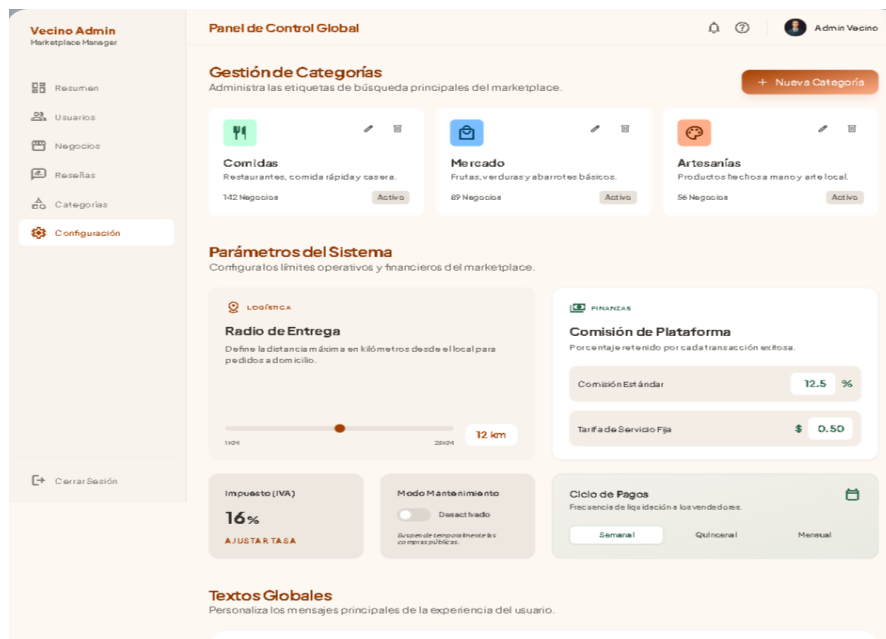
Figura 26: prototipo de alta fidelidad, módulo de geolocalización



Nota. Prototipo interactivo de alta fidelidad correspondiente al módulo de geolocalización. Elaboración propia (2026). Disponible en:

<https://stitch.withgoogle.com/preview/6944331621580144500?node-id=9426e73b54b34f6d9b9c9fca2d70ca16>

Figura 27: prototipo de alta fidelidad, panel del comerciante

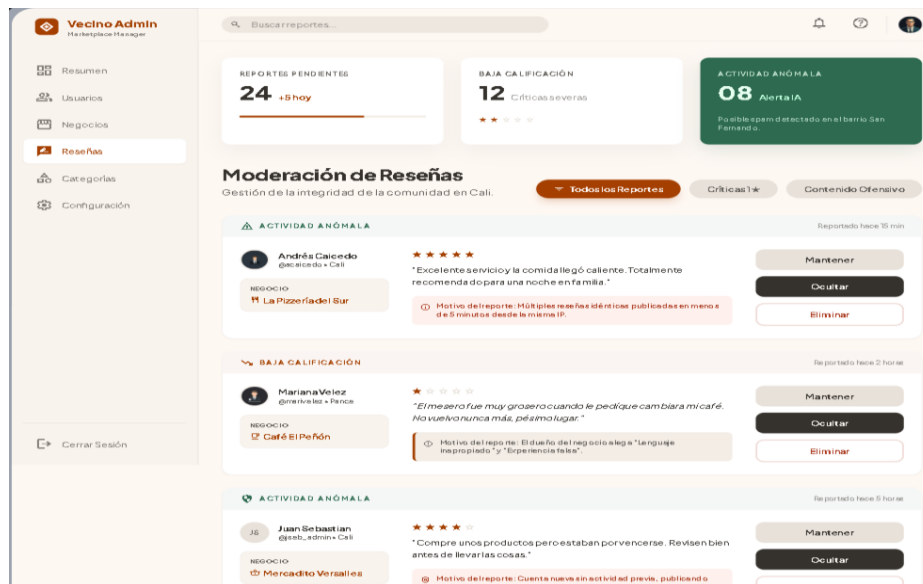


Nota.

Prototipo interactivo de alta fidelidad correspondiente al panel del comerciante Elaboración propia (2026). Disponible en:

<https://stitch.withgoogle.com/preview/6944331621580144500?node-id=0673c7375ac04ca993a7ddad950d3931>

Figura 28: prototipo de alta fidelidad, administración del sistema



Nota.

Prototipo interactivo de alta fidelidad correspondiente administración del sistema. Elaboración propia (2026). Disponible en:

<https://stitch.withgoogle.com/preview/6944331621580144500?node-id=e3a17e63318543fd91220235a8d9fc8e>

Conclusiones

En conclusión, el desarrollo de esta actividad de formulación posibilitó el establecimiento de las bases conceptuales, técnicas y organizativas del proyecto de software llamado Vecino, una plataforma de mercado hiperlocal que busca cambiar la dinámica del comercio local en ciudades medianas colombianas. Durante el proceso, se demostró que la detección precisa de un problema real, por ejemplo, la exclusión digital de los comerciantes pequeños, es el primer paso esencial para crear soluciones tecnológicas relevantes y con gran impacto social.

En definitiva, el proyecto, que consiste en once módulos integrados y abarca el ciclo comercial en su totalidad, está diseñado para satisfacer una necesidad estructural y no una solución superficial. La elección de construir una arquitectura cliente-servidor en tres capas utilizando el stack Node.js, Express, React y PostgreSQL asegura que la plataforma cuente con una base firme, escalable y fácil de mantener, lo cual permitirá su desarrollo paulatino.

En fin, la implementación de la metodología Scrum como marco de gestión del desarrollo posibilitó que el trabajo se organizara de forma estructurada mediante cuatro Sprints con entregables funcionales específicos, designando roles definidos en el equipo y estableciendo las ceremonias requeridas para asegurar una mejora constante del proceso. Esta metodología, apoyada por instrumentos como GitHub y Jira, hace más fácil la colaboración eficiente entre los miembros del equipo y el seguimiento de lo que se desarrolla. También, La representación precisa y profesional de la solución planteada, que se empleará como plan de acción para las fases de implementación a lo largo del semestre, se logra a través del modelamiento del sistema por medio de diagramas de clases, casos de uso, entidad-relación y secuencias. Además, los prototipos elaborados en Stich y Figma también contribuyen a este propósito. En última instancia, este proyecto es una oportunidad para crear una solución tecnológica concreta que fomente la digitalización del comercio barrial, fortalezca el tejido económico local y produzca valor real para las comunidades en las que se llevará a cabo; no sólo es un ejercicio académico.

Referencias

- Cohn, M. (2004). *User stories applied: For agile software development*. Addison-Wesley.
- Institute of Electrical and Electronics Engineers. (2011). *IEEE 830: Recommended practice for software requirements specifications*. IEEE.
- International Organization for Standardization. (2011). *ISO/IEC 25010: Systems and software engineering — Systems and software quality requirements and evaluation (SQuaRE)*. ISO.
- Project Management Institute. (2021). *A guide to the project management body of knowledge (PMBOK® guide) (7.^a ed.)*. PMI.
- GitHub, Inc. (2025). *GitHub: Where the world builds software*. GitHub. <https://github.com>
- Atlassian. (2025). *Jira Software: Plan, track, and release great software*. Atlassian. <https://www.atlassian.com/software/jira>
- Figma, Inc. (2025). *Figma: The collaborative interface design tool*. Figma. <https://www.figma.com>
- Google. (2025). *Stitch by Google: AI-powered UI design tool*. Google. <https://stitch.withgoogle.com>
- OpenJS Foundation. (2024). *Node.js documentation*. OpenJS Foundation. <https://nodejs.org/en/docs>
- Meta Platforms. (2024). *React – A JavaScript library for building user interfaces*. Meta Platforms. <https://react.dev>
- The PostgreSQL Global Development Group. (2024). *PostgreSQL: The world's most advanced open source relational database*. PostgreSQL Global Development Group. <https://www.postgresql.org>

Actividad 4 - Diseño del Prototipo

Santiago José Barbosa Rivas ID 100198965

Jerson Javier Ramírez Ricardo ID 100123048

Mario Alexander Avellaneda Buitrago ID 100180605

José Luis Arias ID 100143942

Facultad De Ingeniería, Programa De Ingeniería De Software

Corporación Universitaria Iberoamericana

Proyecto de Software

Dra. Tatiana Cabrera

03 De Mayo De 2026

Resultado de aprendizaje:

Demostrar la aplicabilidad en la construcción de software funcional, enmarcado en buenas prácticas de principio a fin, aplicando técnicas de programación de acuerdo con las necesidades.

Saberes cognitivos

(Relacionados con la apropiación de conceptos, teorías y procesos que el estudiante debe comprender)

- Ciclo de vida del prototipo: concepto, recolección de requisitos, fases de construcción y refinamiento.

Saberes procedimentales

(Acciones que debe realizar el estudiante para lograr un desempeño adecuado)

- Apropiar e identificar los conceptos y fases para el desarrollo de prototipos funcionales.

Saberes actitudinales

(Acciones relacionadas con la actitud y disposición del estudiante)

- Participar activamente en la construcción del prototipo funcional del proyecto en desarrollo.

Descripción de la actividad

Cordial saludo, estimados estudiantes.

El desarrollo de esta actividad corresponde a la **segunda entrega del proyecto de software**, la cual da continuidad al proceso iniciado en la actividad anterior.

En la fase anterior, los equipos desarrollaron:

- Identificación del problema
- Proceso de Design Thinking
- Prototipos de baja y alta fidelidad
- Pruebas de usabilidad con usuarios

Para esta segunda entrega, **se mantienen los prototipos de alta fidelidad previamente validados**, los cuales servirán como base para el desarrollo funcional.

A partir de estos resultados, el equipo deberá avanzar hacia:

- La definición de la arquitectura del sistema
- La construcción del primer prototipo funcional

El objetivo es transformar el diseño conceptual en una estructura técnica viable, definiendo:

- Arquitectura del sistema
- Componentes tecnológicos
- Organización inicial del código
- Primeras funcionalidades del producto

El desarrollo continuará bajo un enfoque iterativo basado en Sprints.

Entregables de la actividad

1. Actualización del MVP

Debe incluir:

- Ajustes realizados al MVP con base en la retroalimentación
- Mejoras en funcionalidades o experiencia de usuario

Historias de usuario:

- Mínimo 6 historias priorizadas
- Formato:

Como [tipo de usuario]

Quiero [funcionalidad]

Para [beneficio]

Cada historia debe incluir:

- Descripción
- Criterios de aceptación

2. Prototipo funcional del sistema

Debe estar basado en el **prototipo de alta fidelidad**. Tener en cuenta que los prototipos deben ser acordes a los Requisitos funcionales.

Debe incluir:

- Entregar pruebas de usabilidad de los prototipos mínimo 5 usuarios.
- Implementación de interfaces principales

Ejemplos:

- Registro o autenticación
- Visualización de información

- Gestión de contenido
- Interacción básica
- Interfaz alineada con el diseño en Figma

Nota:

- Si las funcionalidades son solo de backend, deben estar documentadas con Swagger.
- Si incluyen frontend, se debe indicar claramente cuáles interfaces fueron desarrolladas.

3. Arquitectura del sistema

Se deben **mantener y presentar los diagramas C4**:

- **Nivel 1 (Contexto)**
- **Nivel 2 (Contenedores)**

Incluyendo:

- Usuarios
- Sistema
- Servicios externos
- Frontend, backend y base de datos

4. Documento ADR

Debe justificar decisiones técnicas como:

- Lenguaje de programación
- Frameworks (React, Express, etc.)
- Tipo de arquitectura (cliente-servidor, API REST, etc.)
- Base de datos
- Herramientas adicionales

5. Repositorio del proyecto

Debe incluir:

- Estructura base del proyecto
- Configuración inicial
- Uso de ramas
- Herramientas de calidad (ESLint, Prettier)

6. Retrospectiva del Sprint

Documento de una página que responda:

- ¿Qué funcionó bien?
- ¿Qué dificultades se presentaron?
- ¿Qué se mejorará en el siguiente Sprint?

Debe estar firmado por todos los integrantes.

Entregable final

Esta entrega es la **continuación de la anterior**. Por lo tanto:

- Se debe anexar la nueva información a la entrega previa
- Incluir una hoja separadora indicando “Segunda entrega”
- Incorporar los ajustes realizados a partir de la retroalimentación anterior
- Añadir los nuevos elementos desarrollados en esta fase

Debe incluir obligatoriamente:

- Dos (2) funcionalidades implementadas
- Un video corto donde se evidencien dichas funcionalidades

Actualización del MVP

1.1 Ajustes realizados con base en la retroalimentación

Con base en la retroalimentación recibida en la primera entrega, el equipo realizó los siguientes ajustes al MVP del sistema Vecino:

- Se consolidó la arquitectura del frontend migrando de React (SPA) a Next.js 16 con App Router, lo que permite mejor rendimiento, SSR y rutas protegidas por rol nativas.
- Se adoptó Supabase como proveedor de autenticación y base de datos, reemplazando la implementación JWT + PostgreSQL manual inicial, reduciendo la complejidad del backend en el Sprint 1.
- Se refinaron los prototipos de alta fidelidad en Figma, alineando los tokens de color, tipografía (Sora + Nunito Sans) y componentes base con el sistema de diseño implementado en código.
- Se actualizó el flujo de recuperación de contraseña para incluir confirmación por correo y redireccionamiento seguro mediante tokens de recuperación de Supabase Auth.
- Se incorporó un sistema de roles (usuario / vendedor) con rutas protegidas independientes (/perfil y /vendedor), permitiendo diferenciación de experiencia desde el primer login.

1.2 Historias de usuario priorizadas (Sprint 1)

A continuación se presentan las seis historias de usuario priorizadas para esta entrega, correspondientes al Sprint 1 del proyecto Vecino. Todas pertenecen al Épico EP-01: Autenticación y usuarios.

Tabla 1: *seis historias de usuario priorizadas*

ID	Como...	Quiero...	Para...	Criterios de aceptación
----	---------	-----------	---------	-------------------------

HU-01	Comerciante	Registrarme en la plataforma seleccionando el rol de vendedor	Crear mi perfil de negocio y comenzar a publicar productos	El sistema valida los campos, crea la cuenta, asigna el rol vendedor y redirige a /vendedor
HU-02	Consumidor	Registrarme en la plataforma seleccionando el rol de usuario	Acceder a la oferta local de productos y negocios cercanos	El sistema crea la cuenta, asigna rol usuario y redirige a /perfil
HU-03	Usuario registrado	Iniciar sesión con correo y contraseña	Acceder de forma segura a mi cuenta y funcionalidades según mi rol	El sistema autentica con Supabase Auth, genera sesión y redirige por rol (/perfil o /vendedor)
HU-04	Usuario registrado	Recuperar mi contraseña si la olvidé	Poder volver a ingresar a mi cuenta sin perder el acceso	El sistema envía un correo con enlace de recuperación. El usuario puede definir nueva contraseña mediante token válido.
HU-05	Usuario registrado	Actualizar mi contraseña desde el enlace de recuperación	Restablecer el acceso a mi cuenta de forma segura	El sistema valida el token, permite ingresar nueva contraseña y redirige al inicio de sesión
HU-06	Usuario autenticado	Cerrar sesión desde mi perfil	Proteger mi cuenta en dispositivos compartidos	El sistema destruye la sesión activa y redirige al login. No es posible acceder a rutas protegidas sin nueva autenticación.

2. Prototipo Funcional del Sistema

Diseño en figma:

Figura 1: registro de usuario

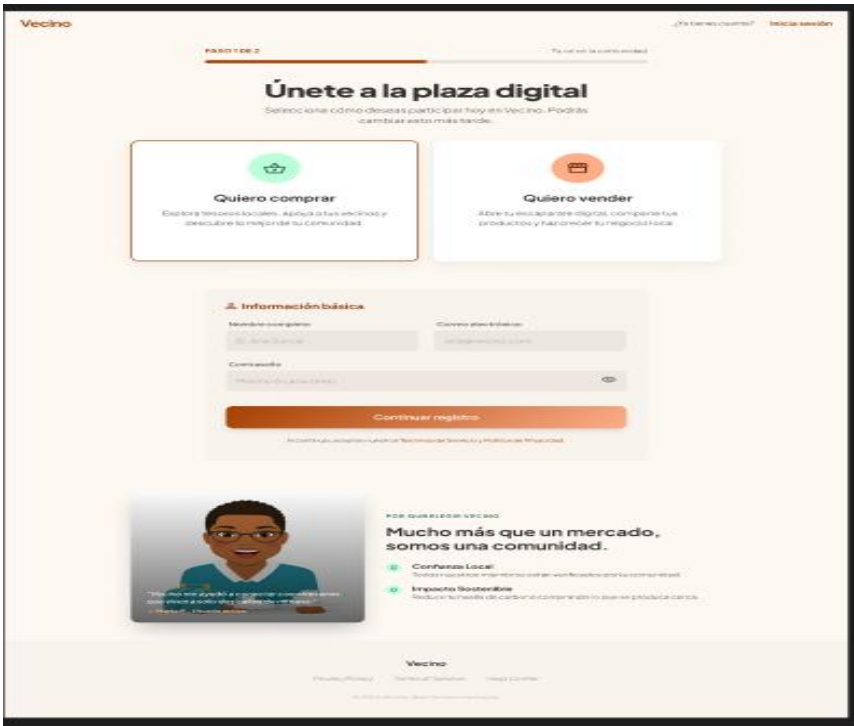


Figura 2: inicio de sesion

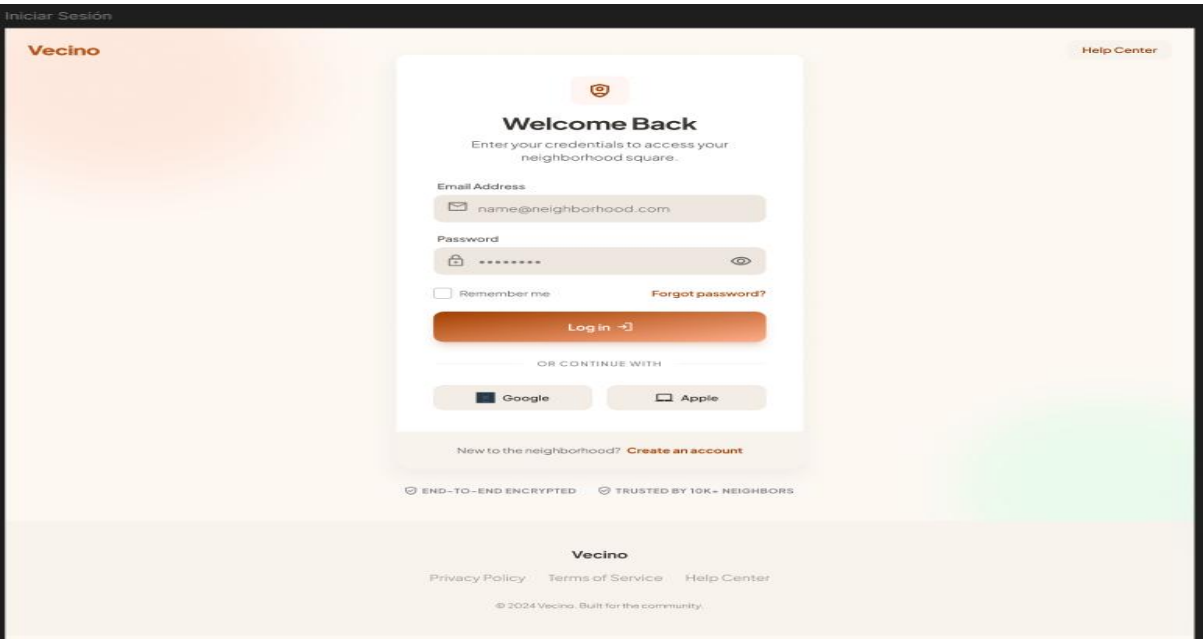


Figura 3: recuperar contraseña

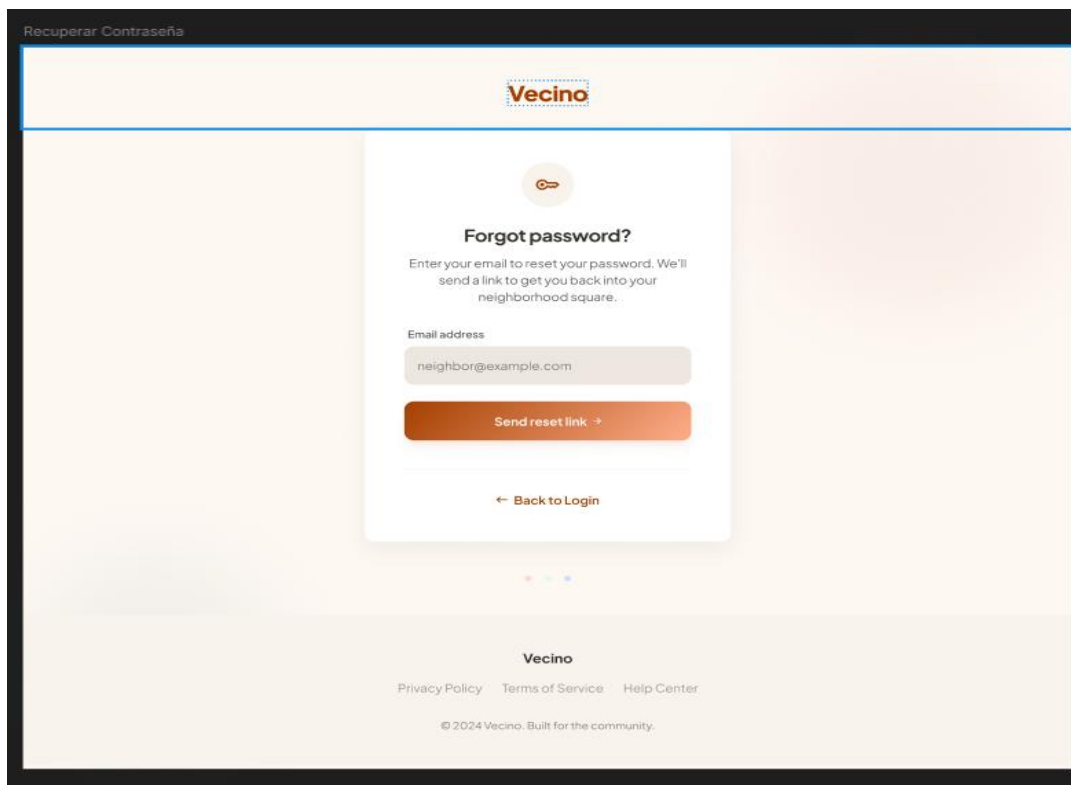


Figura 4: dashboard, inicio publico

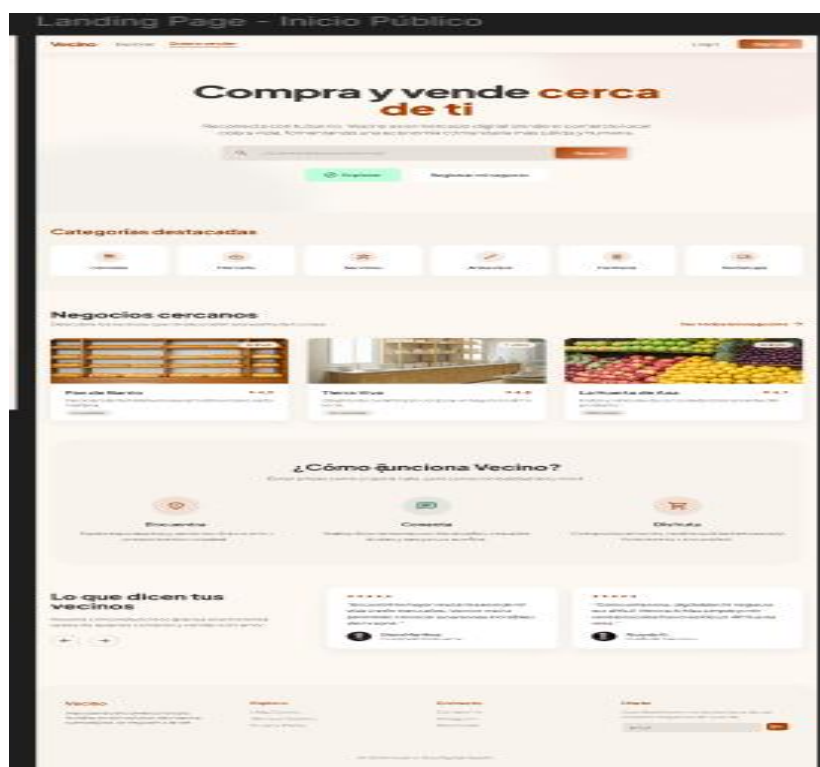


Figura 5: Búsqueda y filtros



Figura 6: Gestion de catalogo

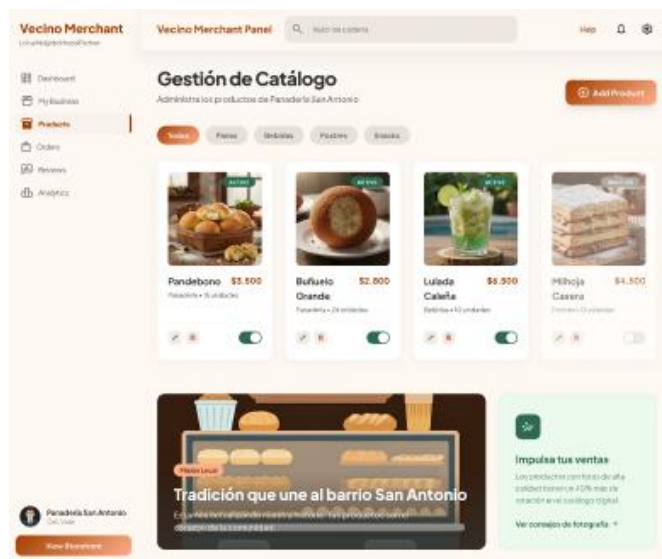


Figura 7: Gestion de perfil

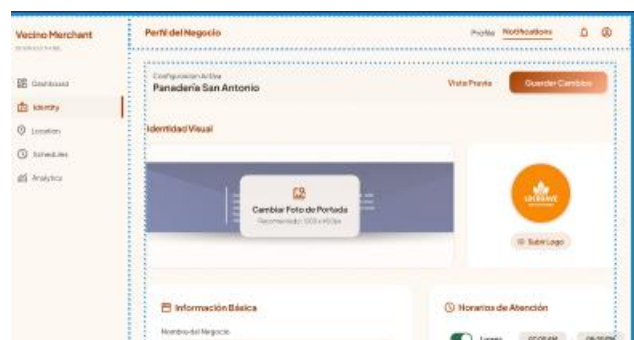


Figura 8: Gestion de compras

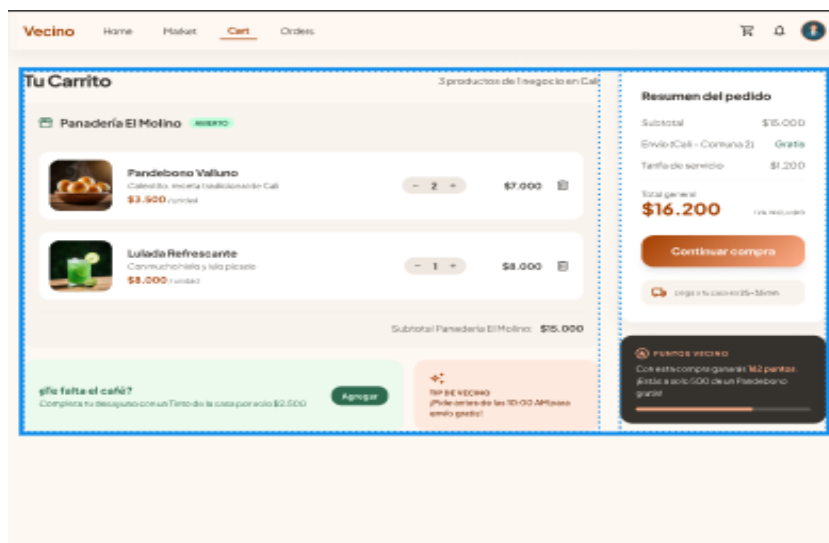


Figura 9: pasarela de pagos



Figura 10: calificar mi pedido

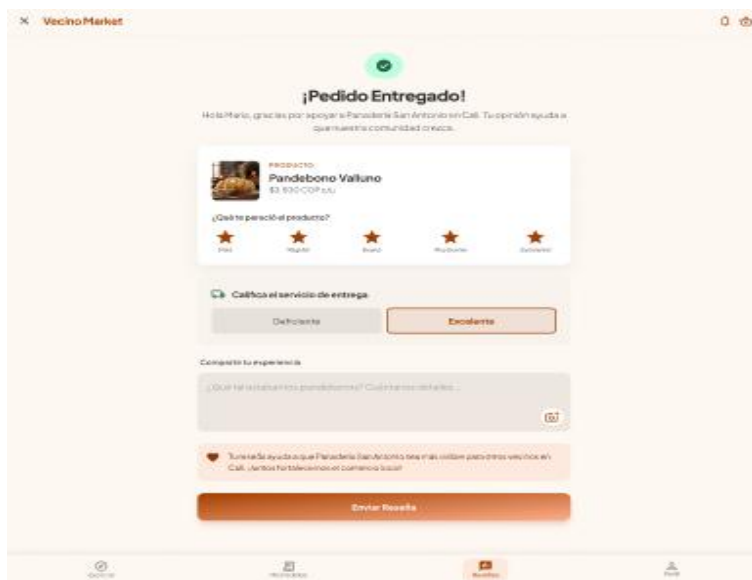


Figura 11: seguimiento de mi pedido



Fig

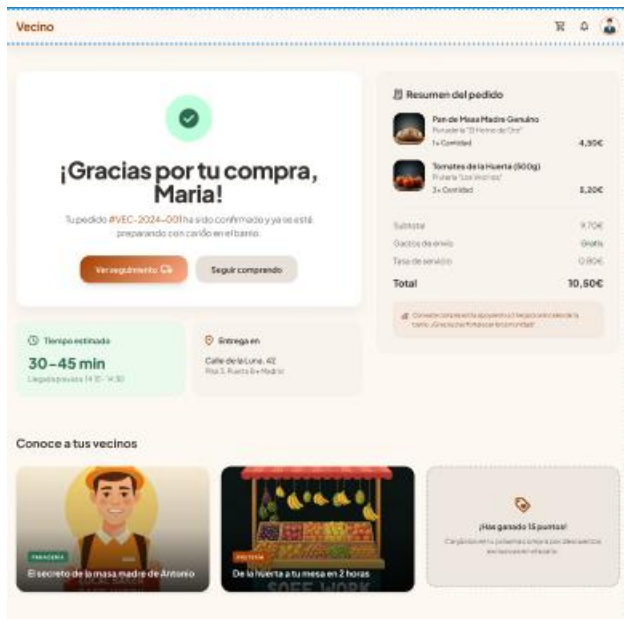


Figura 13: checkout

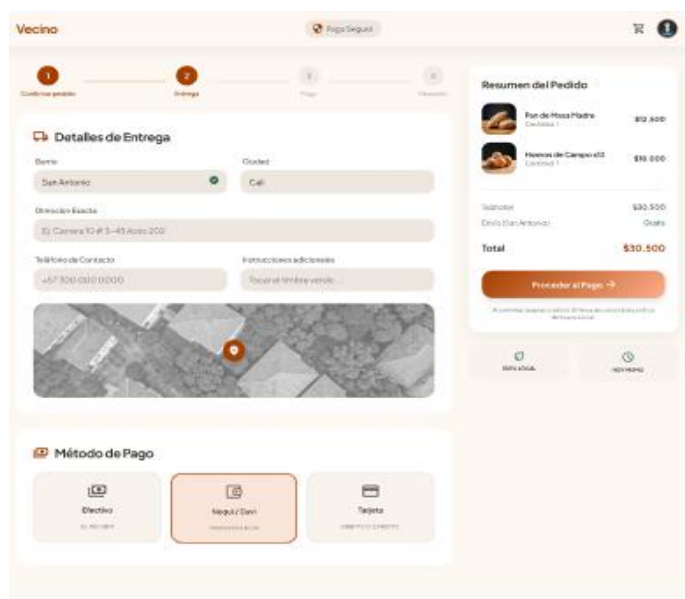


Figura 14: home del consumidor

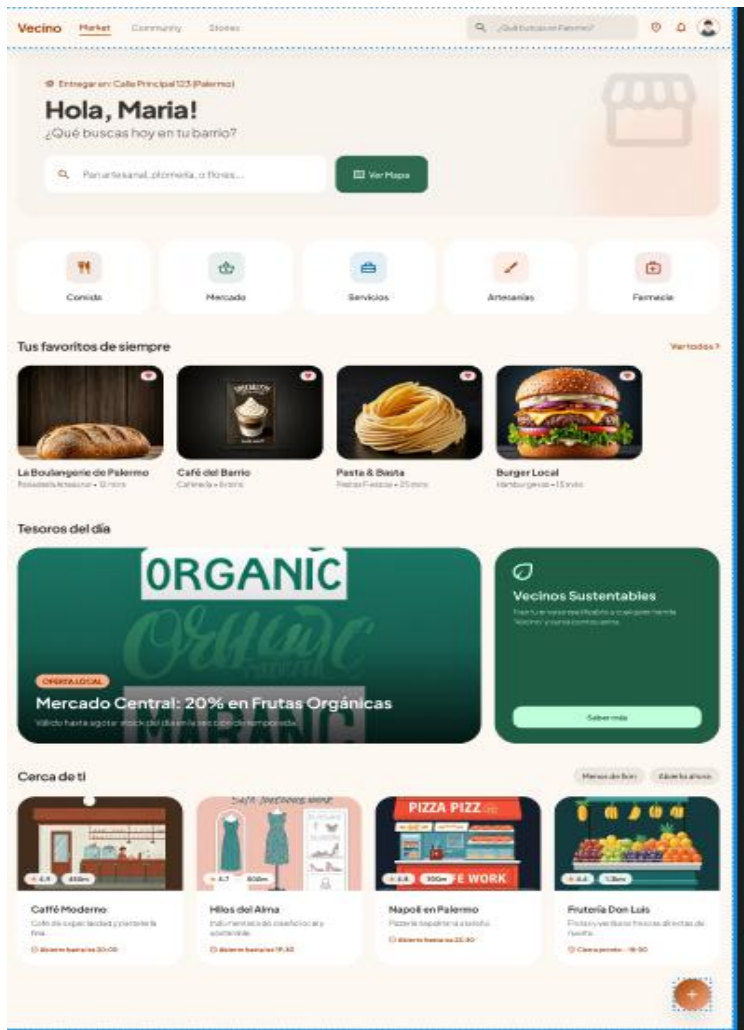
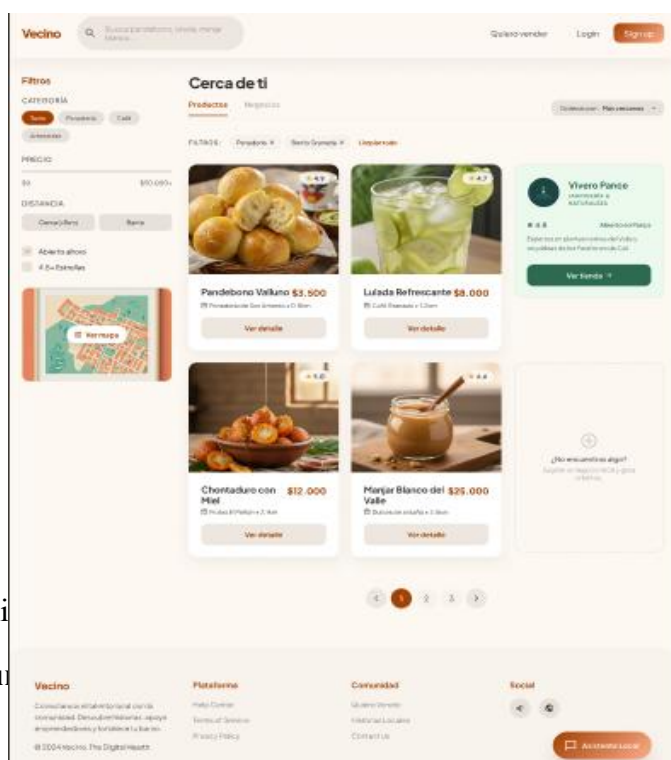


Figura 15: resultados

UI ki

Figura



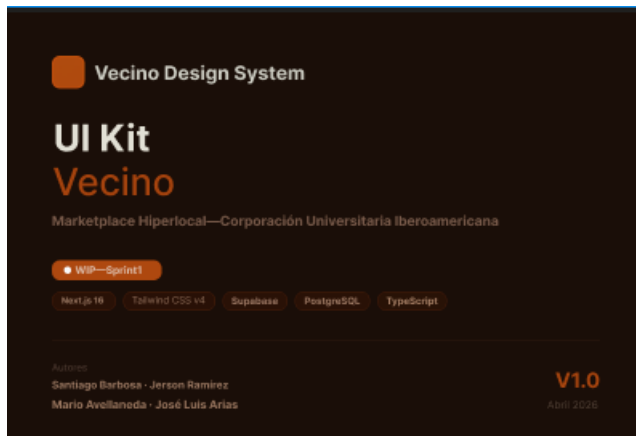
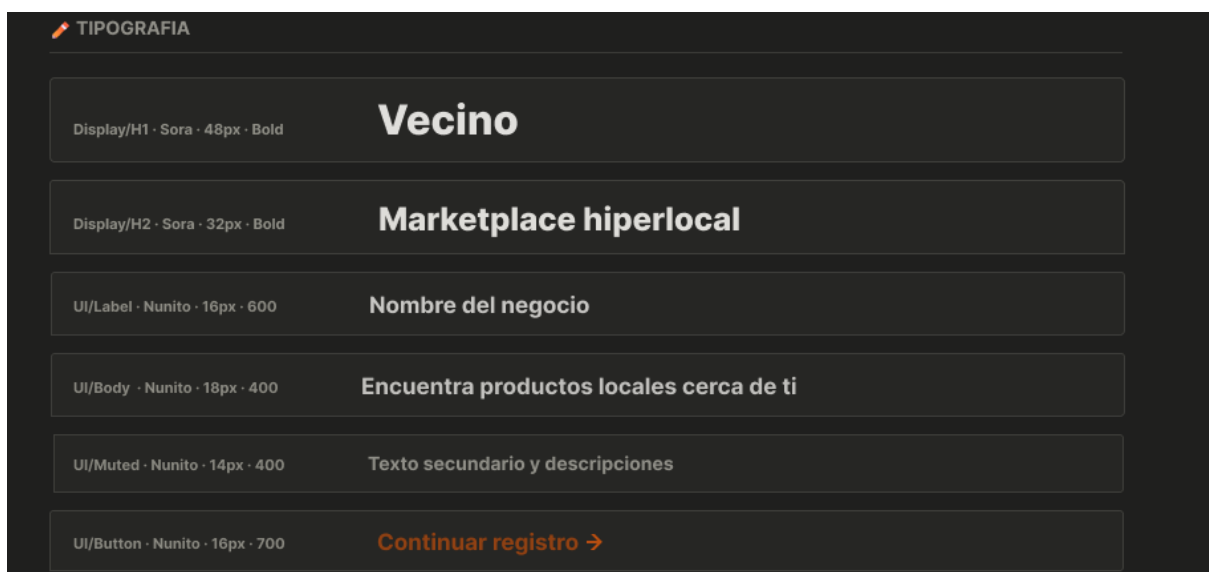


Figura 17: colores tokens vecino



Figura 18: tipografía vecino



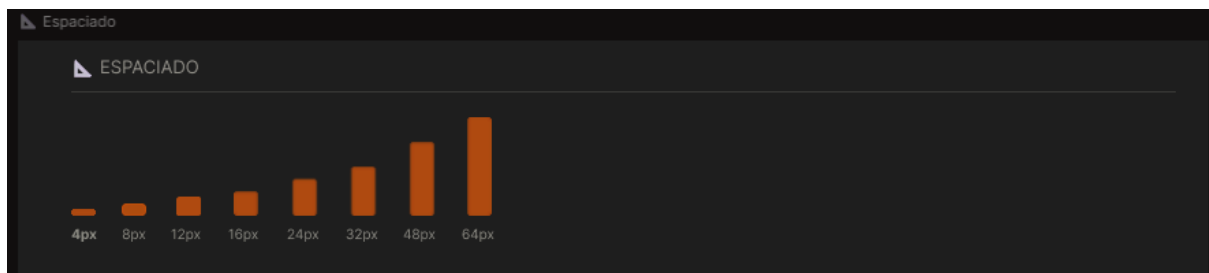


Figura 20: border radius



Figura 21: elevation sombras

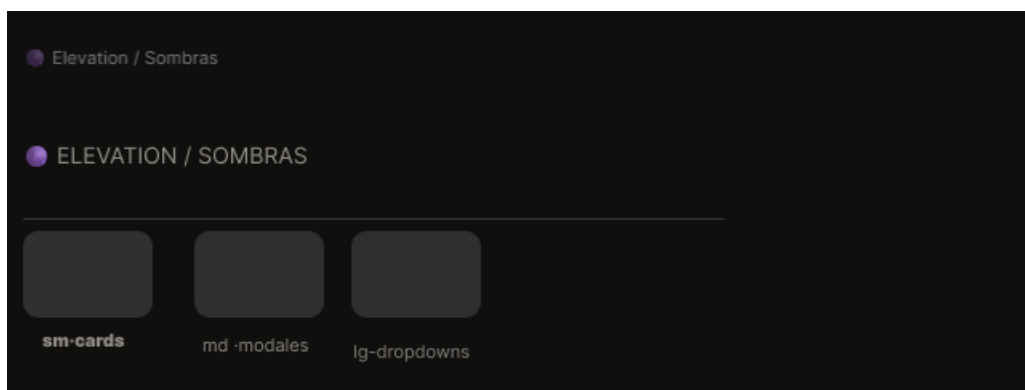
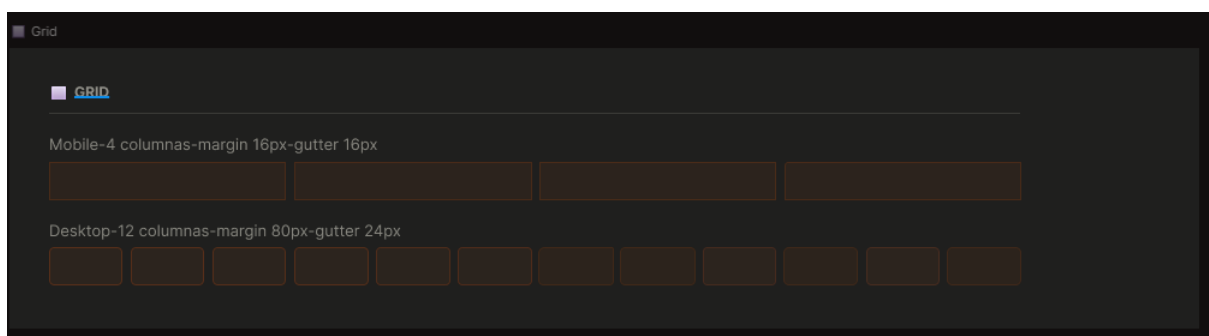


Figura 22: grid



Color tokens

COLOR TOKENS

Token	Valor		Uso
--vecino-brand	#AF4A10		Botones primarios, CTA, acentos
--vecino-brand-strong	#9C3F0C		Hover de botones, estados activos
--vecino-brand-soft	#E29A7e		Fondos suaves, badges secundarios
--vecino-bg	#F6F2EE		Fondo general de la app
--vecino-surface	#F4F4F4		Cards, inputs, contenedores
--vecino-text	#2F2F2F		Texto principal
--vecino-text-muted	#6F6B67		Placeholders, subtítulos
--vecino-border	#D0D8D3		Bordes de inputs y cards
--vecino-error	#BA2D2D		Mensajes de error en formularios
--vecino-success	#B8E8C6		Confirmaciones, estados exitosos

Figura 24: tipografia tokens

Tipografia tokens

TIPOGRAFIA TOKENS

Token	Fuente	Tamaño	Peso
--font-display-h1	Sora	48px	700
--font-display-h2	Sora	32px	700
--font-ui-label	Nunito Sans	16px	600
--font-ui-muted	Nunito Sans	14px	400
--font-ui-body	Nunito Sans	18px	400
--font-ui-button	Nunito Sans	16px	700

Figura 25: espaciado tokens

Espaciado tokens

ESPACIADO TOKENS

Token	Valor	Uso tipico
--space-1	4px	Gap minimo entre elementos
--space-2	8px	Padding de badges, separaciones internas
--space-3	12px	Padding de inputs
--space-4	16px	Padding de cards, gap de formularios
--space-6	24px	Padding de secciones
--space-8	32px	Padding de contenedores
--space-12	48px	Separacion entre secciones

Figura 26: animacion tokens

ANIMACION TOKENS

Frame --duration-fast 150ms Hover, focus	Frame --duration-base 300ms Transiciones UI	Frame --duration-slow 500ms Modales, slides	Button --ease-default ease-in-out General
Frame --ease-bounce cubic-bezier(.34,1.56,.64,1) Botones			

Figura 27: button, variantes y tamaños

● BUTTON-VARIANTES Y TAMAÑOS

Button primary Continuar registro	Button secondary Ver catalogo	Button ghost Cancelar	Button disabled Enviar	Button loading ○ Cargando...
Button md Guardar	Button md (default) Guardar	Button 1g Iniciar sesion		

Figura 28: input estados

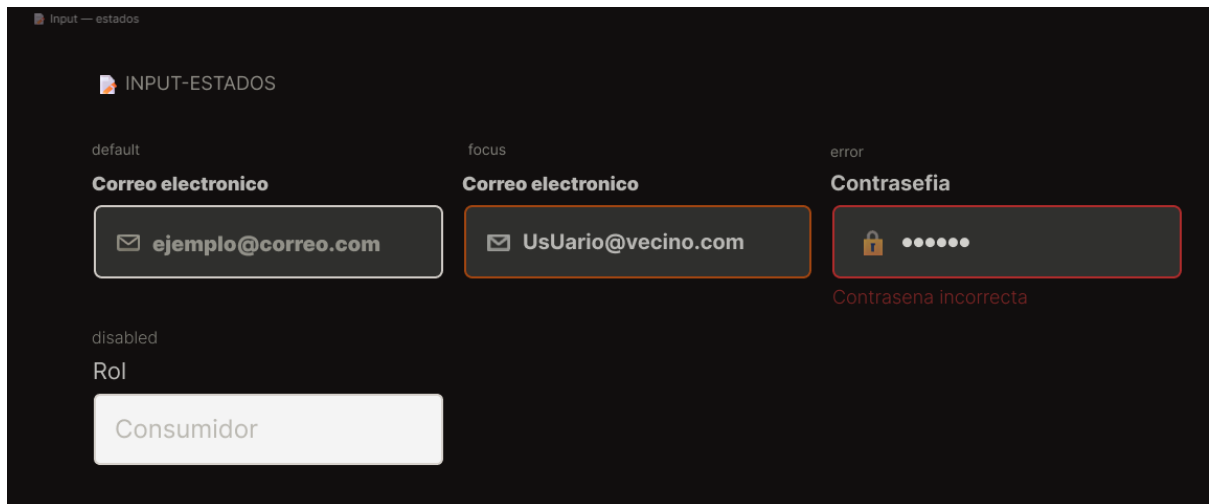


Figura 29: badge/tag

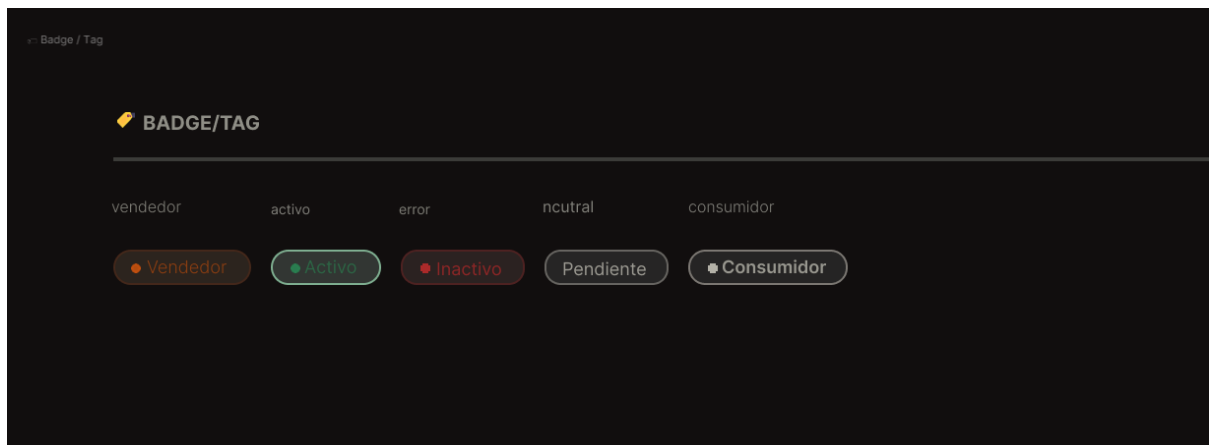


Figura 29: checkbox y radio

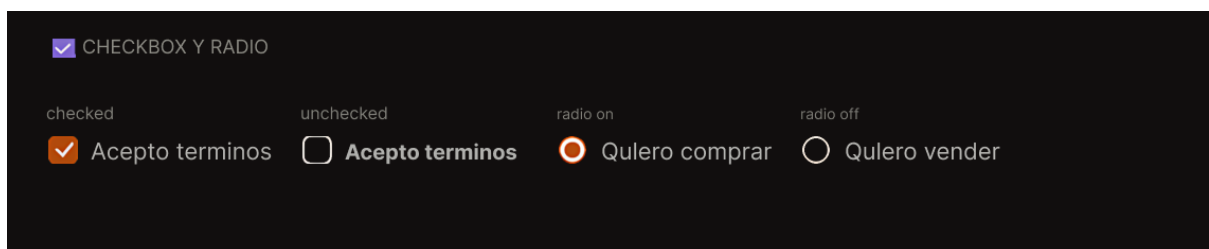


Figura 30: avatar

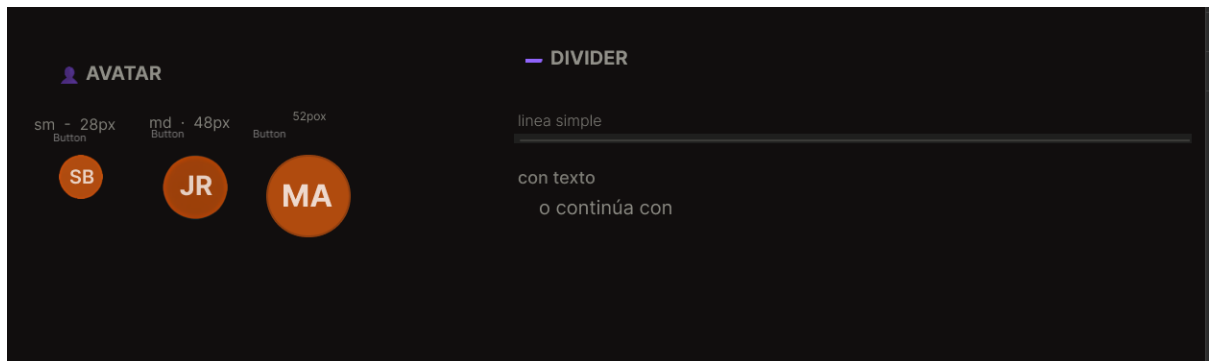


Figura 31: navigation bar

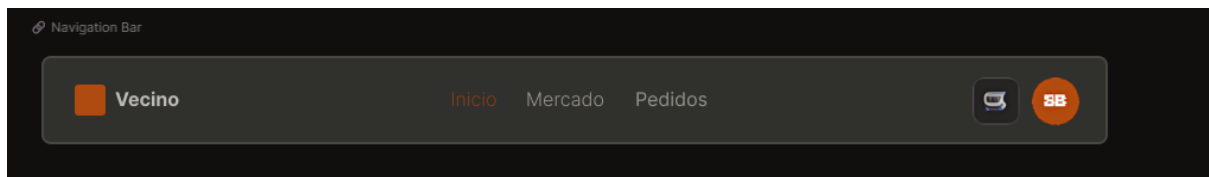


Figura 32: forms login y registro

FORMS-LOGIN Y REGISTRO

Groups

Iniciar sesion
Bienvenido de nuevo a Vecino

Correo electronico
ejemplo@correo.com

Contraseña
00000000

Iniciar sesion

Ovidaste tu contraseña?

No tienes cuenta? [Registrate](#)

Crear Cuenta
Onete alaplaza digital

[Ya Comprar](#) [Vender](#)

Nombre completo
Tu nombre

Correo
ejemplo@correo.com

Contraseña
Minimo 8 caracteres

Continuar registro

Figura 33: card de producto

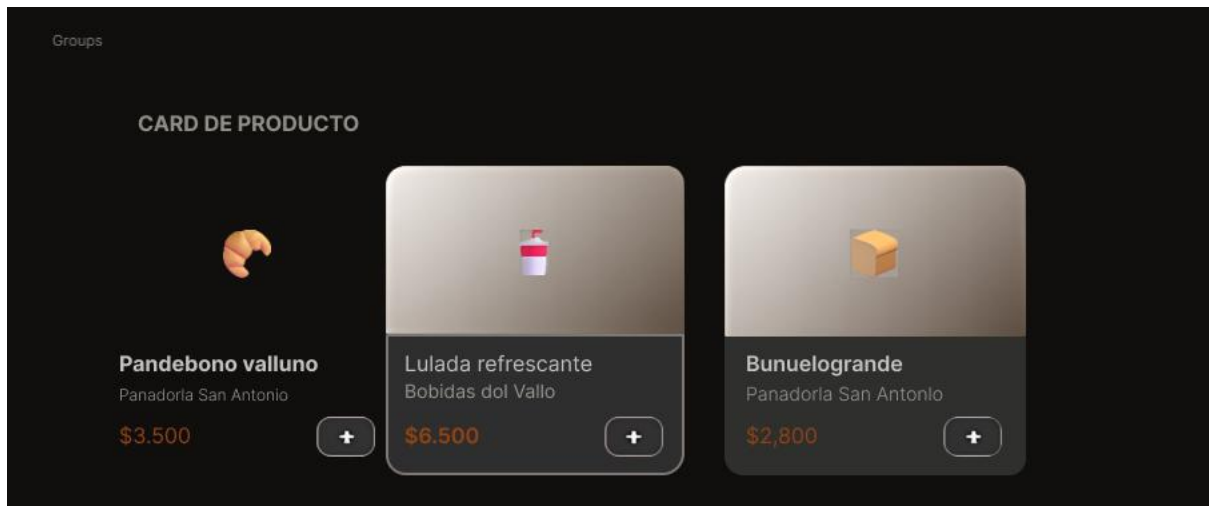


Figura 34: modal de confirmacion

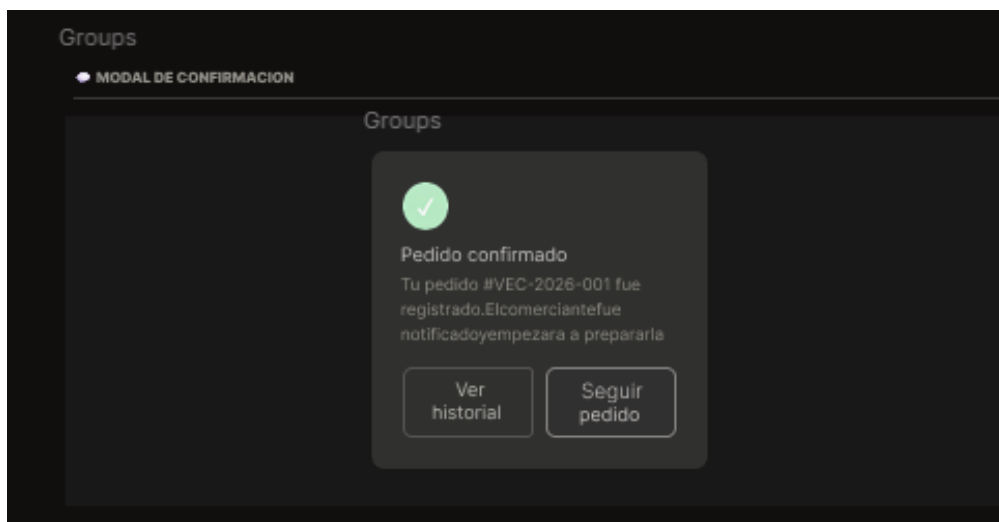


Figura 35: playground

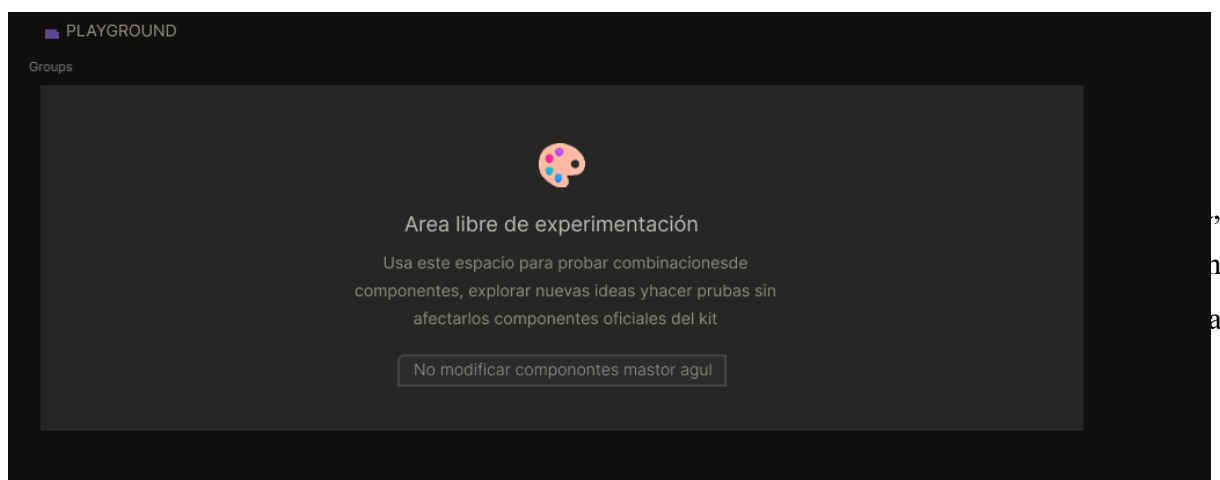


Figura 36: changelog

CHANGELOG			
Groups			
Versión	Fecha	Cambios	Responsable
v1.6	Abril 2026	- Creación inicial del design system - Tokens de color tipografía y espaciado - Foundations: paleta, grid, sombras, radios - Components: Button, Input, Badge, Avatar, Checkbox, Divider - Patterns: Login, Registro, Navbar Card producto, Modal	Santiago Barbosa Jerson Ramírez Mario Avellaneda José Luis Arias
Groups			
v1.7	Próximo sprint	- Componentes de catálogo y carrito - Patrones de bosquejo de filtros - Dark mode tokens	Por definir

Funcionalidad 1, registro de usuario:

Ruta: /registro | Stack: Next.js 16 + Supabase Auth + PostgreSQL

- Formulario con campos: nombre completo, correo electrónico, contraseña y selección de rol (usuario / vendedor).
- Validación de campos obligatorios en frontend con mensajes de error por campo.
- Creación de cuenta en Supabase Auth con metadata de nombre y rol.
- Sincronización automática con la tabla public.usuarios mediante trigger al_crear_usuario_auth.
- Soporte de confirmación por correo electrónico cuando está habilitada en Supabase.
- Redirección automática por rol tras registro exitoso: /perfil (usuario) o /vendedor (comerciante).

Funcionalidad 2 Inicio de sesión:

Ruta: /iniciar-sesion | Stack: Next.js 16 + Supabase Auth

- Formulario con campos: correo electrónico y contraseña.
- Autenticación mediante Supabase Auth (correo + contraseña).
- Gestión de sesión con cookies seguras (cliente y servidor).
- Redirección automática según el rol del usuario autenticado.
- Manejo de errores: credenciales inválidas, cuenta no confirmada, campos vacíos.
- Enlace de navegación hacia recuperación de contraseña.

2.2 Interfaces desarrolladas

Las siguientes interfaces fueron desarrolladas en esta entrega, alineadas con el diseño en Figma:

Tabla 2: interfaces de diseño en figma

Pantalla	Ruta	Descripción
Registro de usuario	/registro	Formulario de creación de cuenta con selección de rol
Inicio de sesión	/iniciar-sesion	Autenticación con email y contraseña, redirección por rol
Recuperar contraseña	/recuperar-contrasena	Envío de correo con enlace de recuperación seguro
Actualizar contraseña	/actualizar-contrasena	Formulario de nueva contraseña tras validación de token
Perfil (consumidor)	/perfil	Ruta protegida para usuarios. Incluye cierre de sesión.
Panel vendedor	/vendedor	Ruta protegida para comerciantes. Incluye cierre de sesión.

2.3 Pruebas de usabilidad

Se realizaron pruebas de usabilidad con 5 usuarios reales sobre los flujos de registro e inicio de sesión. Las pruebas evaluaron claridad de los formularios, comprensión del flujo por rol y facilidad de navegación.

Tabla 3: pruebas de usabilidad con usuarios reales

#	Usuario	Perfil	Flujo probado	Observaciones
1	Usuario 1	Consumidor	Registro + Login	Completó el flujo sin dificultad. Sugirió añadir 'mostrar contraseña'.

2	Usuario 2	Comerciante	Registro como vendedor	Dudó en la selección de rol. Se mejorará la descripción del campo.
3	Usuario 3	Consumidor	Recuperación de contraseña	El flujo fue claro. El usuario encontró el correo de recuperación en la carpeta spam.
4	Usuario 4	Consumidor	Login + cierre de sesión	Completó sin problemas. Tiempo total: menos de 1 minuto.
5	Usuario 5	Comerciante	Registro + acceso a /vendedor	Satisfecho con la redirección automática por rol. Interfaz clara.

3. Arquitectura del Sistema Diagramas C4

3.1 Nivel 1 Diagrama de Contexto

El diagrama de contexto muestra cómo el sistema Vecino interactúa con los actores externos y sistemas de terceros. El sistema actúa como intermediario entre tres tipos de usuarios (consumidor, comerciante, administrador) y se apoya en servicios externos para autenticación, almacenamiento y pagos.

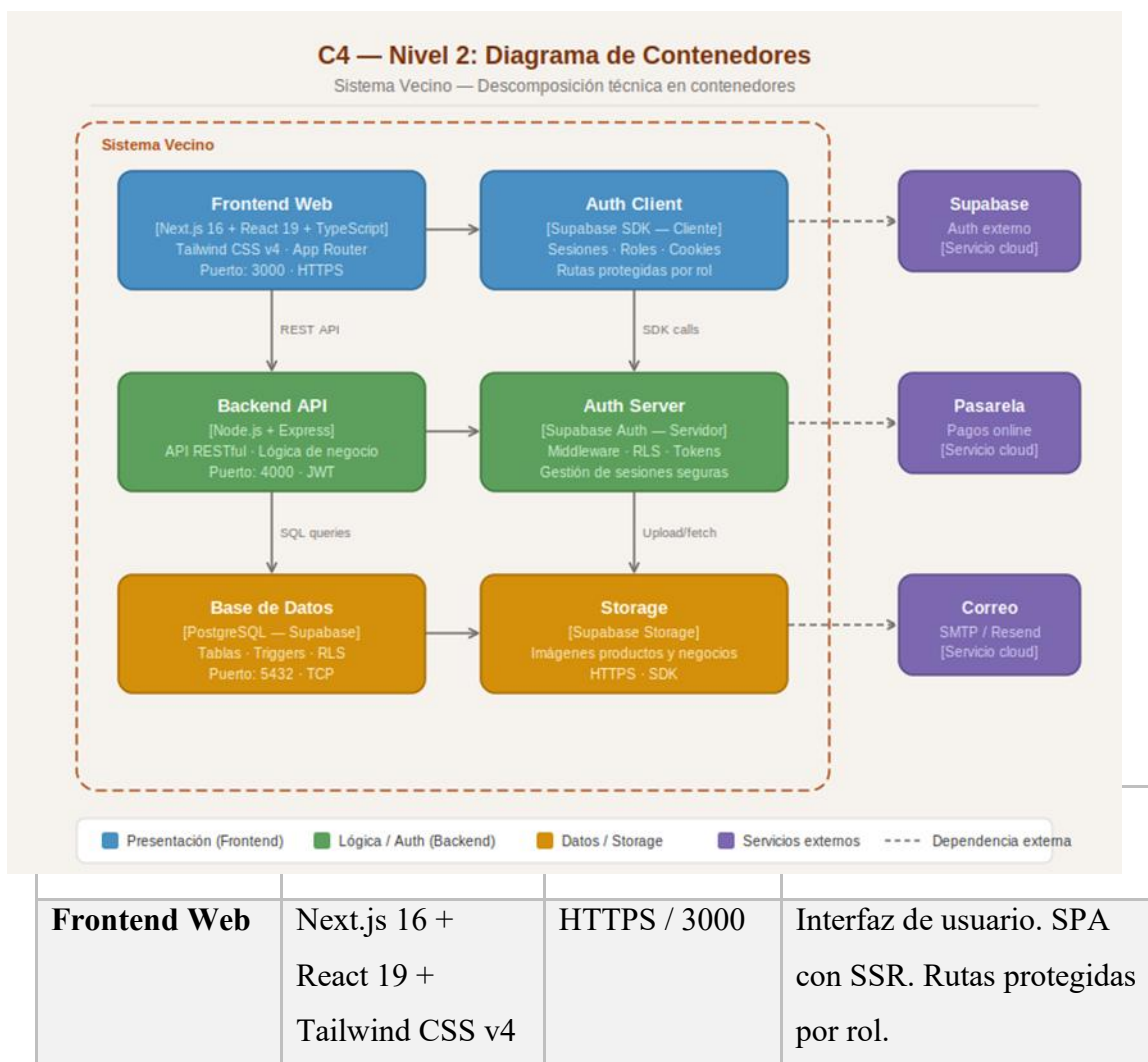
Figura C4-1: Diagrama de contexto del sistema Vecino (Nivel 1)



3.2 Nivel 2 Diagrama de Contenedores

El diagrama de contenedores descompone el sistema Vecino en sus bloques tecnológicos principales, mostrando cómo se comunican entre sí y con los sistemas externos.

Figura C4-2: Diagrama de contenedores del sistema Vecino (Nivel 2)



Backend API	Node.js + Express (planeado)	HTTPS / 4000	API RESTful. Lógica de negocio: pedidos, productos, negocios.
Base de datos	PostgreSQL (Supabase)	TCP / 5432	Almacenamiento relacional. Tablas: usuarios, negocios, productos, pedidos.
Auth Service	Supabase Auth	HTTPS / SDK	Gestión de cuentas, tokens JWT, roles y sesiones seguras.
Storage	Supabase Storage	HTTPS / SDK	Almacenamiento de imágenes de productos y negocios.

4. Documento ADR Architecture Decision Records

Los siguientes registros documentan las decisiones técnicas tomadas por el equipo durante el Sprint 1 del proyecto Vecino, justificando la elección de cada tecnología o patrón arquitectónico.

ADR-01: Framework de Frontend: Next.js 16

CAMPO	DETALLE
ESTADO	Aceptado
CONTEXTO	El proyecto requiere una interfaz web responsiva con rutas protegidas por rol, SSR para rendimiento, y soporte nativo de TypeScript.
DECISIÓN	Usar Next.js 16 con App Router en lugar de React puro (CRA/Vite).

JUSTIFICACIÓN	Next.js provee SSR/SSG nativo, manejo de rutas como sistema de archivos, middleware para protección de rutas, y mejor SEO. El App Router permite layouts por rol sin configuración adicional.
CONSECUENCIAS	Mayor curva de aprendizaje inicial. Se gana en rendimiento, organización y escalabilidad del proyecto.

ADR-02: Autenticación: Supabase Auth

<i>Campo</i>	<i>Detalle</i>
<i>Estado</i>	Aceptado
<i>Contexto</i>	El Sprint 1 requiere autenticación con roles, recuperación de contraseña y sesiones seguras sin construir un backend completo desde cero.
<i>Decisión</i>	Usar Supabase Auth en lugar de implementar JWT + bcrypt manualmente en Node.js.
<i>Justificación</i>	Supabase Auth ofrece gestión de cuentas, tokens JWT, confirmación por correo, recuperación de contraseña y SDK oficial para Next.js, reduciendo el tiempo de desarrollo del módulo de auth en un 60%.
<i>Consecuencias</i>	Dependencia de un servicio externo. El backend propio se construirá en sprints posteriores para la lógica de negocio (pedidos, productos).

ADR-03 Base de datos: PostgreSQL (Supabase)

Campo	Detalle
Estado	Aceptado
Contexto	El sistema requiere almacenamiento relacional para usuarios, negocios, productos y pedidos con relaciones complejas y soporte de roles.
Decisión	Usar PostgreSQL gestionado por Supabase con RLS (Row Level Security) para control de acceso por propietario.

Justificación	PostgreSQL es el motor relacional más robusto y open source. Supabase lo expone con dashboard, migraciones y SDK. RLS permite seguridad a nivel de fila sin lógica adicional en el backend.
Consecuencias	El esquema debe diseñarse cuidadosamente desde el inicio. Las políticas RLS agregan complejidad pero mejoran la seguridad de manera significativa.

ADR-04 Estilos: Tailwind CSS v4

Campo	Detalle
Estado	Aceptado
Contexto	El proyecto requiere un sistema de estilos consistente, responsivo y alineado con los tokens de diseño definidos en Figma.
Decisión	Usar Tailwind CSS v4 con variables CSS personalizadas (tokens Vecino) en lugar de CSS modules o styled-components.
Justificación	Tailwind permite aplicar el design system directamente en JSX, elimina dead CSS, es mobile-first por defecto y v4 usa CSS nativo para tokens sin necesidad de configuración adicional.
Consecuencias	Los tokens Vecino (--vecino-brand, --vecino-bg, etc.) se definen en globals.css y se referencian en Tailwind v4 directamente.

ADR-05 Arquitectura: Cliente-Servidor en tres capas

<i>Campo</i>	Detalle
<i>Estado</i>	Aceptado
<i>Contexto</i>	El equipo necesita una arquitectura clara, separada por responsabilidades, que permita el desarrollo paralelo de frontend y backend.
<i>Decisión</i>	Arquitectura cliente-servidor en tres capas: presentación (Next.js), lógica de negocio (Node.js + Express API REST), datos (PostgreSQL).

<i>Justificación</i>	La separación de capas permite que cada integrante trabaje en su dominio sin bloquear al otro. La API REST es independiente del frontend y puede ser consumida por otras interfaces en el futuro.
<i>Consecuencias</i>	Requiere definir contratos de API (endpoints, payloads) entre el equipo. Se documentarán con Swagger en sprints posteriores.

5. Repositorio del Proyecto

5.1 Estructura del repositorio Frontend

El repositorio del frontend está disponible en:
<https://github.com/BARBOSA191919/Software-Vecino-Frontend>

Tabla 4: carpetas y propósitos del proyecto Vecino

<i>Carpeta / Archivo</i>	Propósito
<i>src/app/(auth)/</i>	Rutas de autenticación: registro, login, recuperación, actualización de contraseña
<i>src/app/(app)/</i>	Rutas protegidas por rol: /perfil (consumidor), /vendedor (comerciante)
<i>src/components/auth/</i>	Componentes reutilizables: AuthInput, AuthPrimaryButton, AuthSurface
<i>src/lib/supabase/</i>	Clientes Supabase: browser, server y proxy para middleware
<i>supabase/schema.sql</i>	Esquema de base de datos: tablas, triggers, RLS y políticas de acceso
<i>mockups/</i>	Referencias visuales de login, registro y recuperación usadas como base de diseño
<i>eslint.config.mjs</i>	Configuración de ESLint para calidad del código TypeScript/React
<i>.env.example</i>	Variables de entorno requeridas: URL de Supabase, clave pública, DATABASE_URL

5.2 Herramientas de calidad configuradas

- ESLint: configurado con `eslint.config.mjs` para TypeScript y React, detecta errores estáticos y malas prácticas.
- Prettier: configurado para formateo automático y consistente del código en todo el equipo.
- TypeScript 5.x: tipado estático en todo el proyecto para reducir errores en tiempo de ejecución.
- `tsconfig.json`: configurado con paths estrictos y soporte de Next.js App Router.

5.3 Uso de ramas

- `main`: rama principal protegida. Solo recibe cambios mediante Pull Request revisado.
- `feature/auth`: rama de desarrollo del módulo de autenticación (Sprint 1).
- `feature/ui-kit`: rama para el sistema de diseño y componentes base.
- `hotfix/*`: ramas para correcciones urgentes en producción.

6. Retrospectiva del Sprint 1

Documento de retrospectiva del Sprint 1 — Sistema Vecino | Fecha: Abril 2026

¿Qué funcionó bien?

- La integración de Supabase Auth con Next.js fue más rápida de lo esperado gracias al SDK oficial y la documentación disponible.
- La separación de componentes (`AuthInput`, `AuthPrimaryButton`, `AuthSurface`) facilitó la reutilización y coherencia visual en todas las pantallas de auth.
- El uso de variables CSS (tokens Vecino) alineó perfectamente el código con el diseño en Figma, reduciendo inconsistencias visuales.
- La gestión del backlog en Jira permitió tener visibilidad clara del avance de cada historia de usuario durante el sprint.
- Las pruebas de usabilidad con 5 usuarios se realizaron de manera fluida y arrojaron observaciones accionables para el siguiente sprint.

¿Qué dificultades se presentaron?

- La configuración inicial del proxy de Supabase para Server Components en Next.js generó conflictos con el middleware que tomaron tiempo adicional en resolver.
- La selección de rol en el formulario de registro no era suficientemente clara para usuarios no técnicos, lo que se evidenció en las pruebas de usabilidad.
- La sincronización del design system entre Figma y el código requirió iteraciones adicionales para alinear los tokens de color exactamente.
- Coordinar los tiempos de entrega entre los integrantes del equipo presentó dificultades por cargas académicas paralelas.

¿Qué se mejorará en el siguiente Sprint?

- Mejorar la descripción del campo de selección de rol en el registro, incluyendo ejemplos o íconos que diferencien claramente los perfiles.
- Implementar la funcionalidad 'mostrar/ocultar contraseña' en todos los campos de tipo password, según sugerencia de usuarios en las pruebas.
- Iniciar la construcción del backend API (Node.js + Express) para los módulos de productos y negocios del Sprint 2.
- Establecer reuniones de sincronización semanales fijas para mejorar la coordinación del equipo.
- Completar la configuración del UI Kit en Figma con variables, text styles y componentes base para guiar el desarrollo de las siguientes pantallas

Link del video:

[actividad 4.MOV](#)

Conclusiones

La culminación de esta segunda fase del proyecto demuestra la viabilidad técnica de transformar un diseño conceptual de alta fidelidad en un prototipo funcional sólido, enmarcado en el cumplimiento de las buenas prácticas de desarrollo y los requisitos funcionales establecidos. A través de la definición de una arquitectura basada en diagramas C4 y la justificación de decisiones tecnológicas mediante el documento ADR, se logró establecer una estructura técnica coherente que garantiza la escalabilidad del sistema. Este proceso de construcción, guiado por un enfoque iterativo de Sprints y la priorización de historias de usuario, permitió no solo implementar las primeras funcionalidades críticas del producto, sino también asegurar la calidad del código mediante el uso de herramientas como ESLint y Prettier en el repositorio. Finalmente, la integración de pruebas de usabilidad con usuarios reales y la realización de una retrospectiva del equipo confirman que el software no solo responde a una necesidad técnica, sino que evoluciona constantemente a partir de la retroalimentación, logrando una alineación precisa entre la experiencia de usuario diseñada en Figma y la interacción real del sistema desarrollado.

Referencias

- **Brown, S. (2018).** *The C4 model for visualizing software architecture*. Recuperado de c4model.com. (Es la fuente oficial para explicar los niveles de Contexto y Contenedores que entregaste).
- **Nygaard, M. (2011).** *Documenting Architecture Decisions*. (Referencia clave para el documento ADR que justifica tu stack tecnológico).
- **Fowler, M. (2014).** *Patterns of Enterprise Application Architecture*. (Ideal para sustentar la arquitectura cliente-servidor o API REST mencionada en tu ADR).
- **Cohn, M. (2004).** *User Stories Applied: For Agile Software Development*. Addison-Wesley Professional. (La base académica para el formato "Como [usuario], quiero [acción], para [beneficio]").
- **Schwaber, K., & Sutherland, J. (2020).** *La Guía de Scrum*. (Sustenta el uso de Sprints y la Retrospectiva que firmó el equipo).
- **Nielsen, J. (1993).** *Usability Engineering*. Morgan Kaufmann. (Referencia clásica para justificar por qué 5 usuarios son suficientes para identificar la mayoría de los problemas de usabilidad).
- **Gothelf, J., & Seiden, J. (2016).** *Lean UX: Designing Great Products with Agile Teams*. O'Reilly Media. (Conecta el proceso de Design Thinking con el desarrollo funcional).